

TECHNICAL DOCUMENT

CodeSpectra

A Local AI-Powered Codebase Intelligence Workbench

An architectural and engineering analysis of
a desktop application for deep codebase understanding.

Trần Lê Minh

steven0410leminh@gmail.com

<https://github.com/StevenTran0410/CodeSpectra>

APRIL 2026

Abstract

CodeSpectra is a local desktop application designed to solve the *codebase onboarding problem* — the significant time cost engineers spend understanding an unfamiliar repository before becoming productive contributors. Existing tools either require cloud data uploads, produce shallow static analyses, or lack the structured depth needed for real engineering workflows.

This report documents the full technical design and implementation of CodeSpectra: a hybrid Electron + Python system combining a lexical-structural retrieval engine, a compiled C++ native extension for graph hotspot computation, and a Haystack-orchestrated pipeline of eleven specialised LLM analysis agents. The system operates entirely on the user’s machine when configured with a local inference server, with no data transmitted unless the user explicitly opts in to a bring-your-own-key cloud provider.

Keywords: codebase intelligence, retrieval-augmented generation, LLM pipeline, static analysis, graph-based retrieval, local-first AI, Electron, Python, C++ native extension.

Acknowledgements

This project makes use of the following open-source projects: Python, FastAPI, Electron, React, Haystack (deepset), Tree-sitter, pybind11, SQLite/aiosqlite, Tailwind CSS, and the broader open-source LLM ecosystem (Ollama, LM Studio). Their maintainers and contributors are gratefully acknowledged.

Table of Contents

Abstract	i
Acknowledgements	ii
List of Figures	ix
List of Tables	xi
List of Abbreviations	xii
1 Introduction	1
1.1 Motivation	1
1.2 Research Objectives	2
1.3 Scope and Limitations	2
1.4 Contributions	4
1.5 Document Structure	5
2 Background and Related Work	6
2.1 Code Intelligence and Developer Tools	6
2.2 Retrieval-Augmented Generation	6
2.2.1 Chunking for Source Code	6
2.2.2 Lexical Retrieval for Code	7
2.3 Large Language Models for Code Understanding	7
2.4 Graph-Based Code Analysis	7

2.5	Pipeline Orchestration Frameworks	7
2.6	Desktop Application Architecture	8
2.7	Summary	8
3	System Architecture	9
3.1	High-Level Overview	9
3.2	Technology Stack	10
3.3	Electron–Python Integration	11
3.3.1	Startup Handshake	11
3.3.2	IPC Communication Pattern	11
3.3.3	Development vs. Production	11
3.4	Domain Model and Database Design	12
3.5	Privacy Architecture	13
3.6	Job and Progress System	13
3.7	Design Decisions and Trade-offs	14
4	Codebase Indexing and Retrieval System	15
4.1	Overview of the Indexing Pipeline	15
4.2	File Discovery and Manifest	16
4.3	Symbol Extraction with Tree-sitter	17
4.3.1	Symbol Normalisation and Deduplication	17
4.4	Structural Graph Construction	18
4.4.1	Import Resolution via Suffix Index	18
4.4.2	Noise Filtering	18
4.4.3	Centrality Scoring	19
4.4.4	Circular Import Detection via SCC	19
4.4.5	Louvain Community Detection	19

4.4.6	Symbol Reference Edge Extraction	20
4.5	AST-Based Semantic Chunking	21
4.5.1	Chunking Pipeline	21
4.5.2	Language Support	21
4.5.3	Target Size by Category	22
4.5.4	Symbol Name Extraction	22
4.6	BM25-Based Retrieval Scoring	23
4.6.1	BM25 Lexical Signal	23
4.6.2	Structural Signal	24
4.6.3	Section-Specific Retrieval Budgets	24
4.7	Two-Stage Retrieval Pipeline	24
4.8	Static Analysis Signals	25
4.8.1	Risk Analysis (<code>static_risk.py</code>)	25
4.8.2	Convention Analysis (<code>static_convention.py</code>)	26
4.9	Index Caching and Skip Logic	26
5	LLM Analysis Pipeline	27
5.1	Pipeline Architecture Overview	27
5.2	Agent Design	28
5.2.1	Base Agent and JSON Repair Loop	28
5.3	Individual Agents	29
5.3.1	Agent A — Project Identity Agent (2 000 tokens)	30
5.3.2	Agent D — Coding Conventions Agent (3 000 tokens)	30
5.3.3	Agent G — Important Files Radar Agent (2 000 tokens)	31
5.3.4	Agent I — Glossary Agent (3 000 tokens)	31
5.3.5	Agent J — Risk / Complexity Agent (3 000 tokens)	31
5.3.6	Agent B — Architecture Overview Agent (2 500 tokens)	31

5.3.7	Agent C — Repository Structure Agent (2 000 tokens)	32
5.3.8	Agent E — Forbidden Patterns Agent (2 000 tokens)	32
5.3.9	Agent H — Onboarding Agent (4 000 tokens)	32
5.3.10	Agent F — Feature Map Agent (5 000 tokens)	32
5.3.11	Agent K — Evidence Auditor Agent (2 000 tokens)	32
5.3.12	Agent L — Synthesis Agent (4 000 tokens)	33
5.4	Haystack Orchestration	33
5.4.1	Component Wrapping	33
5.4.2	Pipeline Execution Dynamics	33
5.5	Pipeline Memory Context	34
5.6	Agent Result Caching	34
5.7	Large-Codebase Mode	35
5.8	JSON Schema Enforcement	35
5.9	Per-Section Rerun	36
5.10	Model Capability Guard	36
6	Implementation Details	37
6.1	Native C++ Extension	37
6.1.1	Function 1: <code>compute_scores</code> — Centrality Scoring	37
6.1.2	Function 2: <code>expand_neighbors</code> — BFS Neighbourhood Expansion	39
6.1.3	Function 3: <code>compute_scc</code> — Iterative Tarjan’s Algorithm	40
6.1.4	Function 4: <code>scan_keywords_bulk</code> — Bulk Keyword Scanning	41
6.1.5	Function 5: <code>compute_louvain</code> — Community Detection	42
6.1.6	Function 6: <code>rank_and_budget</code> — Two-Stage Greedy Selection	42
6.1.7	Build System	42
6.2	Multi-Language Symbol Extraction Architecture	43
6.2.1	Language-Specific Walkers	44

6.2.2	Normalisation and ExtractMode	44
6.3	LLM Provider Adapters	44
6.3.1	Unified Interface	45
6.3.2	Model Discovery	45
6.3.3	API Key Storage	45
6.4	Report Diff System	46
6.4.1	Diff Engine Logic	46
6.5	Interactive Graph Viewer	46
6.5.1	Visualisation Features	47
6.6	Evidence Panel and Section Sources	47
6.7	Frontend Testing with Vitest	47
7	Evaluation and Results	48
7.1	Evaluation Methodology	48
7.2	Retrieval Quality	48
7.3	Agent Output Quality	49
7.4	Performance Benchmarks	50
7.4.1	Indexing Pipeline Performance	50
7.4.2	Native C++ vs Python Fallback	51
7.4.3	LLM Pipeline Timing	52
7.5	Discussion	52
7.5.1	Strengths	52
7.5.2	Weaknesses and Limitations	53
8	Conclusion and Future Work	54
8.1	Summary of Contributions	54
8.2	Future Work	55

8.3	Concluding Remarks	55
A	Agent JSON Output Schemas	57
A.1	Agent A — Project Identity	57
A.2	Agent B — Architecture	57
A.3	Agent C — Structure	58
A.4	Agent D — Conventions	58
A.5	Agent E — Violations	59
A.6	Agent F — Feature Map	60
A.7	Agent G — Important Files	60
A.8	Agent H — Onboarding	61
A.9	Agent I — Glossary	61
A.10	Agent J — Risk	61
A.11	Agent K — Auditor	62
A.12	Agent L — Synthesis	63
B	Backend API Reference	64
B.1	Analysis Endpoints	64
B.2	Repository and Snapshot Endpoints	66
B.3	Provider Configuration Endpoints	68
B.4	Retrieval Debug Endpoints	69
	References	71

List of Figures

3.1	Three-tier desktop architecture of CodeSpectra. All inter-tier traffic is confined to the loopback interface; no public network connection is required at runtime.	9
4.1	Four-stage indexing flow used to transform a repository into a searchable retrieval index.	15
5.1	Execution flow of the multi-agent LLM pipeline, from pre-fetch through the dependent execution waves. Arrows denote information produced at one stage and passed forward as context to downstream agents.	27

List of Tables

2.1	Key technical components of the CodeSpectra architecture.	8
3.1	Core technology choices and rationale.	10
3.2	Database schema organised by conceptual group (schema version 23). .	12
3.3	Privacy modes and data-flow summary.	13
4.1	Target chunk sizes by file category.	22
4.2	Token retrieval budgets per analysis section.	24
5.1	Agent summary: execution wave, token budget, and inter-agent data dependencies.	30
5.2	Components of the pre-fetched <code>PipelineMemoryContext</code>	34
6.1	Execution time of centrality scoring on synthetic dependency graphs. .	38
6.2	Execution time for bulk keyword scanning (300 source files, 4 keywords). .	42
6.3	Sub-modules of the <code>repo_map</code> package.	43
7.1	Retrieval precision comparison on CodeSpectra repository (20-query set). .	48
7.2	Agent output quality scores (human evaluation, 1–5 scale).	49
7.3	Indexing pipeline timing on CodeSpectra repository (>300 files).	50
7.4	C++ native vs Python performance (500-node graph / 300-file keyword scan).	51
7.5	LLM pipeline timing on CodeSpectra repository (GPT-4o-mini).	52
B.1	Analysis API endpoints.	64

B.2	Local repository endpoints.	66
B.3	Snapshot preparation endpoints.	67
B.4	Manifest and repo-map endpoints.	67
B.5	Provider and model-connector endpoints.	68
B.6	Graph, community, and retrieval endpoints.	69
B.7	Utility endpoints (/api/app, /api/consent, /api/workspace, /api/job).	70

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
BFS	Breadth-First Search
BYOK	Bring Your Own Key
CI	Continuous Integration
CPU	Central Processing Unit
C++	C Plus Plus (native programming language)
DAG	Directed Acyclic Graph
DB	Database
DFS	Depth-First Search
GPU	Graphics Processing Unit
IPC	Inter-Process Communication
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
ORM	Object-Relational Mapping
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SCC	Strongly Connected Component
SQL	Structured Query Language

TF-IDF Term Frequency–Inverse Document Frequency

UI User Interface

URL Uniform Resource Locator

Chapter 1

Introduction

1.1 Motivation

Walking into a large, unfamiliar codebase is one of the most consistently underestimated costs in software engineering. Whether a developer is **onboarding** to a new enterprise project or **returning** to a codebase after months away, figuring out the project structure, finding core files, and understanding logic connections takes up a massive amount of time. Re-learning or manually exploring these structures is a slow and frustrating bottleneck.

Existing tools only partially solve this friction. Standard searches like **grep** rely on **exact text matches**, ignoring **semantic intent**. Meanwhile, manually pasting snippets to web LLMs lacks **broader context**. Even modern AI-powered IDEs (e.g., Cursor) often struggle with comprehensive repository retrieval, frequently **missing relevant files** or context during complex queries. Developers need a reliable system that truly understands the entire project.

Finally, the push for AI in software development brings up a major roadblock: **data privacy**. Many companies and enterprise teams have strict security policies and simply cannot upload their proprietary code to public cloud models. CodeSpectra was built to address this with a **"local" approach**. While it may not be the first local AI tool, it is designed to keep code entirely on the user's machine by default. By running indexing locally—while still offering the flexibility to connect to custom, trusted API endpoints if desired—CodeSpectra provides deep codebase intelligence without compromising **data security**.

1.2 Research Objectives

The primary objectives of this project are:

1. Design and implement a **hybrid retrieval engine** that combines lexical search with structural graph signals to identify the most relevant code context for a given analysis task, without relying on dense vector embeddings or external vector databases.
2. Build a **pipeline of specialised LLM analysis agents** (11 agents, covering project identity, architecture, conventions, risk, glossary, onboarding, and synthesis), each with a fixed JSON output schema, orchestrated via a dependency-aware asynchronous pipeline.
3. Develop a **native C++ extension** for performance-critical graph computations (centrality scoring, SCC detection, keyword scanning) that would otherwise become bottlenecks on large repositories.
4. Deliver the system as a **local-first desktop application** (Electron + Python) with no mandatory cloud dependency, supporting both local inference servers and bring-your-own-key cloud providers.
5. Evaluate the system's retrieval quality, agent output consistency, and end-to-end performance on real-world repositories.

1.3 Scope and Limitations

In Scope

CodeSpectra targets the following problem space:

- **Supported languages:** Python, JS/TS, Go, Java, Rust, and C/C++ via Tree-sitter, with a regex fallback for other languages.
- **Repository types:** Local folders and Git-cloned repositories, analyzed as point-in-time snapshots.
- **Repository size:** Optimized for tens to a few thousand files. Memory and analysis time scale with size; no hard ceiling enforced.

- **Analysis output:** Structured reports covering architecture, conventions, risks, onboarding, and domain glossary.

Out of Scope

The following features are explicitly excluded:

- **Code modification:** Strictly read-only analysis; no code generation, editing, or patching.
- **Real-time analysis:** Analyzes fixed snapshots. No live file watching; requires manual re-triggering.
- **Conversational interaction:** Produces predefined reports without an ad-hoc chat interface.
- **Binary artifacts:** Strictly ignores non-text assets (binaries, images) during indexing.

Future Work

Planned expansions to evolve the tool into an interactive workbench include:

- **Generative features:** Suggesting code completions, patches, and execution plans from task descriptions.
- **Interactive Conversational AI:** A chat interface for real-time, semantic querying over the repository context.

Known Limitations

- **Output quality is model-dependent.** The accuracy, completeness, and coherence of each agent section are directly bounded by the capability of the selected LLM. Small models (below approximately 7 billion parameters) frequently produce incomplete, hallucinated, or malformed outputs even with the JSON repair loop in place. The model capability guard issues a warning for known small models, but does not prevent analysis.

- **Non-deterministic outputs.** LLM inference is non-deterministic at temperature > 0 . Two runs on the same repository with the same model may produce different report content. The report diff feature (Section 6.4) is provided to surface meaningful changes between runs.
- **Lexical retrieval ceiling.** The hybrid retrieval system relies on term overlap between agent queries and code tokens. Queries expressed at a semantic level that is distant from the literal identifier names in the codebase will miss relevant context. No embedding-based fallback is available in the current version.
- **No cross-repository analysis.** Each analysis job operates on a single repository snapshot. Microservice architectures or monorepos that span multiple independently versioned repositories are not jointly analysed.
- **Platform dependency for native module.** The C++ native extension must be compiled on the target machine. On platforms where a compatible C++ toolchain is not installed, the extension is unavailable and the affected features (graph hotspot scoring, SCC-based circular import detection, bulk keyword scanning) degrade gracefully to slower Python equivalents or are skipped.

1.4 Contributions

The concrete technical contributions of this work are as follows:

1. **Hybrid lexical-structural retrieval without dense embeddings.** A scoring model combining BM25 term overlap with file-level dependency graph centrality. This lightweight approach requires no GPU, vector database, or embedding model, while remaining highly interpretable and independently tunable per agent.
2. **Boundary-aware chunk expansion algorithm.** Detects mid-function chunk boundaries using brace-balancing (C-family) or indentation analysis (Python) and expands them to clean boundaries. This prevents LLM hallucinations caused by reasoning over incomplete function contexts in code-focused RAG.
3. **A pipeline of eleven specialised LLM analysis agents with fixed output schemas.** A multi-agent system where each agent retrieves specific context and outputs strict JSON. Robustness is ensured via a 3-attempt repair loop with schema hints, while versioned reporting maintains stability and diffability across model updates.

4. **Haystack AsyncPipeline orchestration with cross-agent context sharing.** Expresses agent dependencies as a DAG for parallel execution. A shared `PipelineMemoryContext` eliminates redundant database queries by prefetching frequently accessed context across all agents.
5. **Native C++ extension for performance-critical graph operations.** Uses `pybind11` to offload heavy operations—such as centrality scoring, BFS expansion, Tarjan’s SCC detection, and bulk keyword scanning—preventing Python bottlenecks on large repositories.
6. **Two-level result caching with explicit invalidation.** Implements both index-level (parsing, graph extraction) and agent-level (report payload) caching to avoid redundant processing, combined with a `force_rerun` flag for explicit cache invalidation.
7. **Report comparison and per-section rerun.** Provides a `compare_reports` endpoint to diff report versions via section hashing and field-level deltas. A `rerun_section` endpoint allows isolated agent re-execution using embedded static caches, bypassing full pipeline runs.

1.5 Document Structure

The remainder of this document is organised as follows:

- **Chapter 2** reviews related work and background theory, covering code intelligence tools, RAG systems, and LLM-based code analysis.
- **Chapter 3** presents the overall system architecture, technology stack, and key design decisions.
- **Chapter 4** describes the codebase indexing and retrieval subsystem, including Tree-sitter symbol extraction, chunking, and the hybrid scoring model.
- **Chapter 5** details the LLM analysis pipeline: agent design, Haystack orchestration, memory context sharing, and JSON schema enforcement.
- **Chapter 6** documents key implementation details, engineering challenges, and the C++ native extension.
- **Chapter 7** presents evaluation methodology and results.
- **Chapter 8** concludes with a summary of contributions and future directions.

Chapter 2

Background and Related Work

2.1 Code Intelligence and Developer Tools

Developer tooling spans static analysis (e.g., SonarQube), code navigation (e.g., Sourcegraph), and AI-assisted generation (e.g., GitHub Copilot). While these tools excel at itemized rule checking, search, or inline coding, they lack holistic repository understanding. CodeSpectra bridges this gap. It is not an IDE or completion tool; rather, it orchestrates LLM agents over a hybrid-retrieved context to generate comprehensive, standalone architecture and onboarding reports directly from source code.

2.2 Retrieval-Augmented Generation

Canonical RAG relies on dense vector indices, which are computationally expensive for local-first applications and ignore structural code properties. CodeSpectra eschews dense embeddings for a lightweight hybrid lexical-structural model focusing on:

- **Efficiency:** CPU-friendly lexical retrieval without GPU requirements.
- **Interpretability:** Transparent scoring using BM25, path affinity, and graph centrality.
- **Structural Awareness:** Prioritizing architectural hubs via file dependency graphs.

2.2.1 Chunking for Source Code

Standard text chunking breaks code syntactically, causing LLM hallucinations. CodeSpectra uses a **boundary-aware chunk expansion** algorithm to ensure structural

integrity. It merges C-family chunks based on brace balance and Python chunks via indentation tracking, stopping only at clean functional boundaries.

2.2.2 Lexical Retrieval for Code

To optimize BM25 for code, CodeSpectra splits identifiers (e.g., `userProfile` into `user`, `profile`) for cross-language normalization. It combines term overlap with file-path affinity, domain-category matching, and a 3 : 1 in/out-degree graph centrality score.

2.3 Large Language Models for Code Understanding

CodeSpectra maintains a model-agnostic architecture optimized for reliability:

- **JSON Schema Enforcement:** A 3-attempt repair loop corrects malformed outputs, enabling smaller models to produce strict JSON.
- **Context Management:** Uses per-agent token budgets tuned for information density rather than blindly filling large context windows.

2.4 Graph-Based Code Analysis

Source code forms directed dependency graphs. CodeSpectra exploits this to identify architectural debt and critical modules:

- **Centrality Scoring:** A native C++ extension calculates a weighted composite of in-degree and out-degree to rank file importance.
- **Cycle Detection:** Tarjan’s algorithm identifies Strongly Connected Components (SCCs) in linear time, flagging circular imports for risk assessment.

2.5 Pipeline Orchestration Frameworks

Coordinating 11 specialized agents requires robust orchestration. CodeSpectra uses Haystack’s `AsyncPipeline` (a Directed Acyclic Graph model) to manage inter-dependencies efficiently:

- **Concurrent Execution:** Agents run in parallel as dependencies are met.
- **Shared Context:** A `PipelineMemoryContext` injects pre-fetched metadata globally, eliminating redundant database queries.
- **Standardized Agents:** Subclassed components handle boilerplate (repair loops, LLM dispatch) while isolating analytical logic.

2.6 Desktop Application Architecture

Designed as a local-first, offline application, CodeSpectra pairs an Electron frontend (React) with an embedded FastAPI backend (Python).

- The Node.js main process spawns and monitors the Python FastAPI subprocess.
- Communication uses standard HTTP and Server-Sent Events (SSE) for real-time streaming, governed by a strict OpenAPI contract.

2.7 Summary

CodeSpectra shifts from inline coding assistance to generating structured knowledge documents. Key technical stances include:

Component	Technical Implementation
Retrieval	Hybrid lexical/structural (no vector DB).
Chunking	Boundary-aware expansion to prevent LLM hallucinations.
Orchestration	Haystack <code>AsyncPipeline</code> for DAG-based execution.
Intelligence	11 LLM agents with strict JSON schemas and repair loops.
Performance	Native C++ extension for heavy graph and scanning tasks.

Table 2.1: Key technical components of the CodeSpectra architecture.

Chapter 3

System Architecture

3.1 High-Level Overview

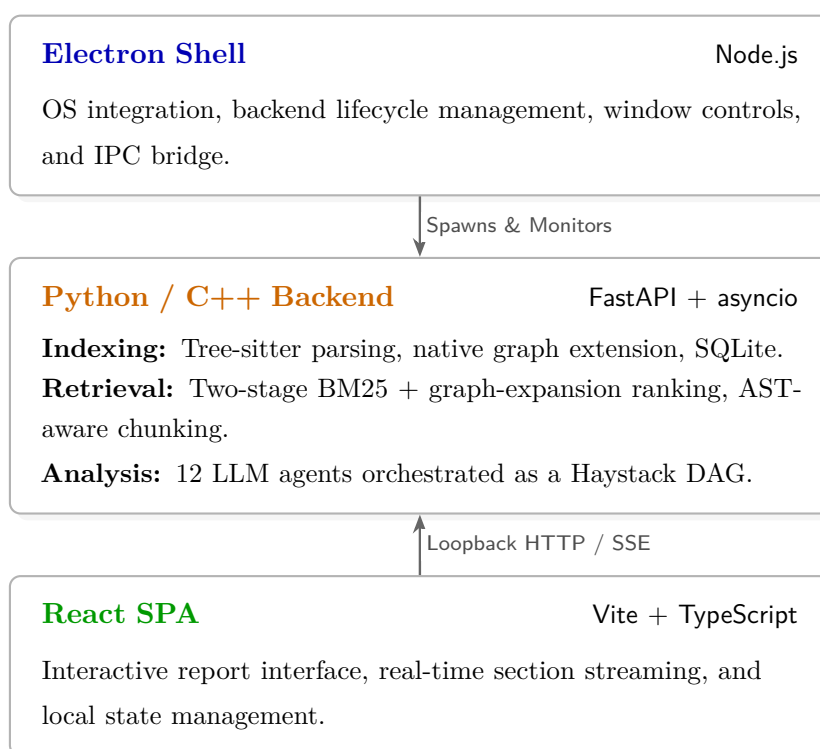


Figure 3.1: Three-tier desktop architecture of CodeSpectra. All inter-tier traffic is confined to the loopback interface; no public network connection is required at runtime.

CodeSpectra follows a three-tier desktop architecture in which each tier runs as a separate operating-system process and communicates with its neighbours through a well-defined interface. The Electron shell is intentionally thin — it spawns the Python server, bridges OS-level access, and routes IPC calls to HTTP. All analysis logic lives in the Python backend. The React renderer knows nothing about file-system paths or model providers; the Python server knows nothing about windowing or Electron APIs.

This strict separation means a fatal error in the analysis engine does not crash the UI process, and the Python server can be restarted independently without requiring the user to close the application window.

3.2 Technology Stack

Table 3.1: Core technology choices and rationale.

Layer	Technology	Rationale
Desktop shell	Electron 28, Node.js 20	Cross-platform desktop host with native dialogs and keychain access
UI renderer	React 18, Vite, TypeScript	Fast, modular UI development
State management	Zustand	Lightweight shared state with minimal boilerplate
Styling	Tailwind CSS	Consistent utility-first styling
Backend API	Python 3.11, FastAPI, Uvicorn	Async API stack with strong typing and OpenAPI support
Database	SQLite, <code>aiosqlite</code> [1]	Embedded storage with WAL-based concurrent reads
LLM orchestration	Haystack <code>AsyncPipeline</code> [2]	DAG orchestration with typed component interfaces
Symbol extraction	Tree-sitter [3]	Incremental multi-language AST parsing
Native acceleration	C++, <code>pybind11</code> [4]	Faster graph and keyword-processing hot paths
Package management	<code>uv</code> , <code>npm</code>	Reproducible lockfiles and fast installs

Python was selected for the backend due to easibility and community support. Tree-sitter’s Python bindings expose the full C API — including pattern matching and node-field access — required for robust multi-language symbol extraction. `pybind11` integrates directly with the CPython C API to produce a standard importable shared library without the debuggability problems of a WebAssembly path. Haystack’s `AsyncPipeline` is Python-only and maps the twelve-agent dependency graph directly onto DAG edges.

3.3 Electron–Python Integration

3.3.1 Startup Handshake

During launch, Electron dynamically allocates a free loopback port and passes it to the Python backend. The main process waits up to 20 seconds for the backend to emit a `BACKEND_READY:{port}` sentinel to `stdout` before resolving the startup promise and launching the UI. Graceful shutdown is managed via Electron’s `before-quit` hooks, which send a `SIGTERM` (or equivalent kill signal on Windows) to terminate the Uvicorn process cleanly.

3.3.2 IPC Communication Pattern

Following Electron’s strict security model, a preload script exposes a typed `window.api` object to the React renderer via `contextBridge`.

Renderer calls trigger `ipcRenderer.invoke()`. The Node.js main process acts as an IPC-to-HTTP proxy, forwarding these invocations to the FastAPI backend and returning the JSON response. HTTP errors are directly re-thrown as JavaScript exceptions.

For long-running analysis jobs, the frontend retrieves real-time agent completion events using cursor-based polling (`GET /api/analysis/events/{job_id}?from_idx=N`). This ensures exactly-once event delivery without relying on complex WebSocket setups.

3.3.3 Development vs. Production

The backend initialization branches based on Electron’s execution context:

- **Development:** Uses a local Python virtual environment (`.venv`) to apply code changes instantly without rebuilds.
- **Production:** Executes a self-contained PyInstaller binary bundled within the application resources.

In both environments, Electron injects the `CODESPECTRA_DATA_DIR` environment variable at spawn time, ensuring the SQLite database is written to the correct OS-specific user data directory.

3.4 Domain Model and Database Design

CodeSpectra stores persistent state in a local SQLite database (`codespectra.db`). Schema evolution is handled by an append-only migration system running on startup. The database operates in **WAL (Write-Ahead Logging) mode**, which is critical for preventing `asyncio` read queries from blocking during long-running write transactions (e.g., bulk chunk inserts).

Table 3.2: Database schema organised by conceptual group (schema version 23).

Group	Tables and Purpose
Configuration	<code>app_metadata</code> , <code>workspaces</code> (with <code>description</code>), <code>provider_configs</code> , <code>github_accounts</code>
Repository State	<code>local_repos</code> , <code>repo_snapshots</code> (immutable point-in-time state)
Indexing Artifacts	<code>manifest_files</code> (UNIQUE on <code>snapshot_id</code> , <code>rel_path</code>), <code>code_symbols</code> , <code>repo_maps</code> , <code>structural_graph_edges</code> , <code>retrieval_chunks</code> , <code>retrieval_indexes</code> , <code>retrieval_bm25_stats</code>
Graph Analysis	<code>graph_community_members</code> , <code>graph_community_summaries</code> (Louvain community detection), <code>symbol_graph_edges</code> (symbol-level call references)
Reports & Jobs	<code>jobs</code> (tracks background execution), <code>analysis_reports</code> (stores JSON outputs)

Completed analysis reports are stored as versioned JSON blobs (currently v3) in the `analysis_reports` table. To support stable diffing and UI consistency, the JSON structure consists of:

- **sections:** The structured outputs from all LLM agents.
- **section_hashes:** SHA-256 digests of each section, enabling the diff engine to fast-path comparisons.
- **static_cache:** Embedded pre-LLM data (risk reports, conventions, graph summaries) to maintain UI consistency even after repository re-indexing.

3.5 Privacy Architecture

CodeSpectra supports two mutually exclusive inference modes, recorded in `analysis_reports.privacy_mode` and enforced at the API layer before any LLM call is issued.

Table 3.3: Privacy modes and data-flow summary.

Mode	Inference target	Data leaving device
Strict Local	Ollama / LM Studio (loopback or LAN)	None; default mode
BYOK Cloud	OpenAI, Anthropic, Gemini, DeepSeek	Code context

Because BYOK Cloud mode transmits potentially proprietary source code to an external API, the application enforces an explicit consent gate before the first cloud call. Consent is stored as `cloud_consent_given` in `app_metadata` and written by a dedicated `POST /api/consent/cloud` endpoint; a disclosure dialog is surfaced in the UI via the `consent:cloud:check` and `consent:cloud:give` IPC channels. API keys are stored in `provider_configs`, never written to log files, never transmitted over IPC beyond the loopback boundary, and not included in exported report files.

3.6 Job and Progress System

Every long-running operation in CodeSpectra is modelled as a *job*. A job row in the `jobs` table carries a JSON `steps` column, allowing different job types to define flexible step sequences without requiring schema migrations:

```

1 {
2   "CLONE":    { "status": "done",    "progress": 100, "message": null },
3   "SYNC":    { "status": "done",    "progress": 100, "message": null },
4   "MANIFEST": { "status": "done",    "progress": 100, "message": null },
5   "PARSE":   { "status": "done",    "progress": 100, "message": null },
6   "GRAPH":   { "status": "done",    "progress": 100, "message": null },
7   "EMBED":   { "status": "running", "progress": 61, "message":
8     "Building index..." },
9   "GENERATE": { "status": "pending", "progress": 0, "message": null },
10  "EXPORT":   { "status": "pending", "progress": 0, "message": null }
11 }
```

Listing 3.1: Example `steps` column for a full analysis job.

Each step's `status` transitions through `pending` | `running` | `done` | `failed` | `skipped`, with `progress` tracking from 0 to 100.

Background execution is managed via `asyncio.Task`, held in a module-level set to prevent premature garbage collection. Cancellation relies on per-job `asyncio.Event` flags checked at defined execution checkpoints. Meanwhile, section-completion events from the Haystack pipeline are buffered in-memory and delivered to the frontend via cursor-based polling (Section 3.3.2).

3.7 Design Decisions and Trade-offs

Desktop application over a web application. Analysing repositories requires deep file-system traversal and Git subprocess execution, which browser-based File System APIs restrict. A local desktop application guarantees this access while strictly satisfying the zero-ops, privacy-first design goal without a persistent cloud server.

Python over Node.js. Critical NLP and parsing libraries (Tree-sitter, Haystack) are Python-centric. Furthermore, Python's `asyncio` effectively handles I/O-bound LLM orchestration without the complexity of multi-threading.

SQLite over a client-server database. CodeSpectra is a single-user tool. SQLite in WAL mode safely supports concurrent `asyncio` reads and writes. As a zero-admin, portable file, it eliminates the need to install and manage a background daemon like PostgreSQL.

Hybrid retrieval over a vector database. Generating dense vector embeddings is prohibitively slow on standard CPUs lacking discrete GPUs. By combining BM25 keyword scoring with structural graph signals, CodeSpectra achieves highly interpretable retrieval (stored natively in SQLite) while avoiding the operational overhead of a standalone vector database.

Haystack over LangChain or LlamaIndex. Haystack's `AsyncPipeline` models workflows as explicit Directed Acyclic Graphs (DAGs). This natively maps to CodeSpectra's multi-agent architecture, automatically resolving dependencies and parallelizing independent agents—a capability cumbersome to implement with LangChain's sequential chains or LlamaIndex's query-focused structures.

Chapter 4

Codebase Indexing and Retrieval System

4.1 Overview of the Indexing Pipeline

Before any language-model query can be answered, CodeSpectra must transform an arbitrary project directory into a structured, searchable representation. Figure 4.1 summarises the four-stage indexing flow.

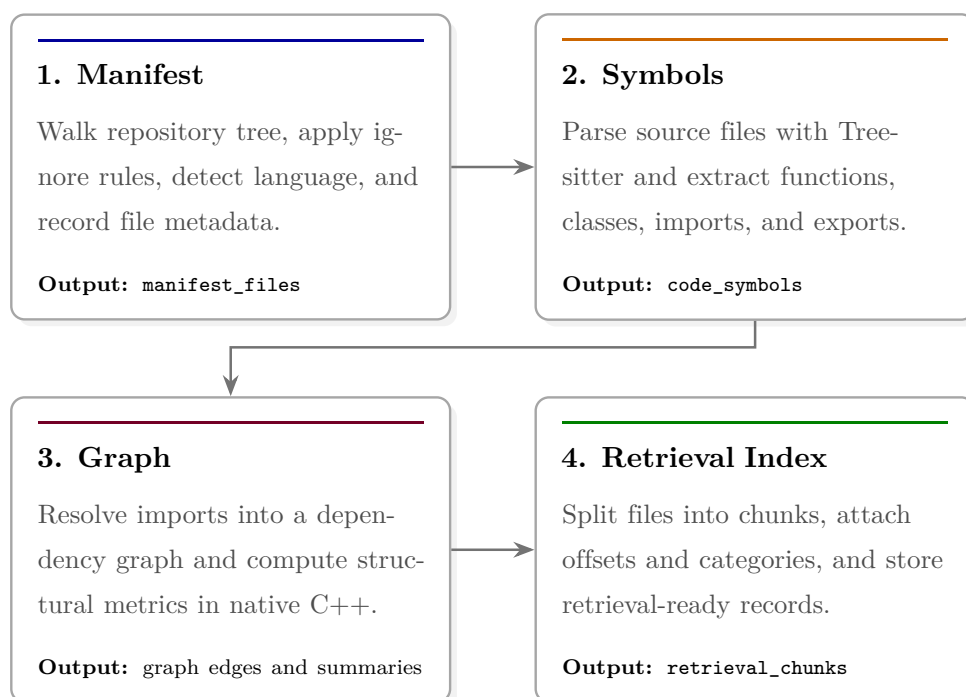


Figure 4.1: Four-stage indexing flow used to transform a repository into a searchable retrieval index.

The figure highlights the main data products created at each stage:

- **Manifest construction** creates the authoritative file inventory used by every later stage.
- **Symbol extraction** converts source files into a normalised symbol table for cross-file reasoning.
- **Structural graph construction** resolves imports into file-level dependencies and computes graph metrics.
- **Retrieval index construction** packages the repository into chunk-level records that the retrieval layer can rank efficiently.

4.2 File Discovery and Manifest

File discovery begins with a depth-first traversal of the project root. At each node the traversal consults a configurable ignore list that combines patterns from `.gitignore`, a built-in blocklist of generated artefact directories (`node_modules`, `__pycache__`, `.venv`, `dist`, `build`), and any project-specific overrides supplied through the CodeSpectra configuration file. Binary files are excluded by checking magic bytes in the first 8 KB of each file.

For each surviving file the manifest records:

- **Relative path** from the project root, used as the canonical identifier throughout the pipeline.
- **Detected language**, resolved by mapping the file extension against a lookup table that covers Python, JavaScript, TypeScript, C, C++, Rust, Go, Java, configuration formats (YAML, TOML, JSON), Markdown, and several others.
- **File category**, a coarser grouping used by the retrieval layer: `source`, `test`, `config`, `docs`, `infra`, or `other`.
- **Byte size** and **last-modified timestamp**, used by the incremental re-index skip logic described in Section 4.9.

Manifest construction also performs a shallow parse of recognised package-manifest files — `package.json`, `pyproject.toml`, ... — to extract project name, version, declared dependencies, and entry-point hints. This metadata is surfaced directly to the LLM pipeline as `manifest_files` in the `PipelineMemoryContext` (Section 5.5). Each discovered file is stored as a row in the `manifest_files` database table, keyed by `snapshot_id`, which represents a logical point-in-time version of the indexed codebase.

4.3 Symbol Extraction with Tree-sitter

CodeSpectra utilises Tree-sitter [3] to parse source code into concrete syntax trees and extract **Symbol** records (name, kind, line spans, dotted `qualified_name`) into the `code_symbols` table. Any language with a community-maintained Tree-sitter grammar package is supported out of the box. The current walker registry covers **28 languages**:

- **Core:** Python, JavaScript, TypeScript, C, C++, Go, Rust, Zig, Bash
- **JVM:** Java, Kotlin, Scala, Groovy
- **Scripting:** Ruby, PHP, Lua, C#
- **Functional:** Haskell, OCaml, Elixir, Julia
- **Config / Data:** YAML, TOML, JSON, CMake, SQL
- **Markup / Styling:** HTML, CSS, Markdown, Svelte

For languages without a Tree-sitter grammar, a regex-based fallback (`_walkers_regex.py`) provides best-effort extraction using common keywords (`def`, `function`, `class`, `fn`).

4.3.1 Symbol Normalisation and Deduplication

After per-language extraction, a normalisation pass reconciles differences in naming and qualification across languages:

1. Import paths are canonicalised to forward-slash relative paths and resolved against the manifest. Symbols that cannot be resolved to a file in the manifest are retained in the table but marked with `resolved = False`; they do not produce graph edges.
2. Kind strings are normalised to a common vocabulary: `function`, `class`, `method`, `import`, `export`, and `type`.
3. Duplicate symbols arising from re-exports or multiple parse passes are deduplicated by the composite key (`snapshot_id`, `file_path`, `qualified_name`, `kind`).
4. The extraction mode (`HYBRID` or `LEXICAL`) is recorded per file. `HYBRID` indicates that Tree-sitter produced the AST; `LEXICAL` indicates that the regex fallback was used and symbol boundaries may be less precise.

4.4 Structural Graph Construction

The structural graph $G = (V, E)$ is a directed graph where each vertex $v \in V$ is a file in the manifest and each edge $(u, v) \in E$ indicates that file u imports or otherwise depends on file v . Edges are derived from the resolved import symbols collected during symbol extraction and are stored as rows in the `structural_graph_edges` table. Aggregate statistics over the graph are stored in `structural_graph_summaries`: node count, edge count, entrypoint files (files with no incoming edges), and the top centrality-ranked files.

To improve database throughput on large repositories, all edge and community-member insertions are now performed via `executemany()` batch calls rather than individual `await db.execute()` per row, eliminating hundreds of async round-trips per graph build.

4.4.1 Import Resolution via Suffix Index

A recurring difficulty in Python codebases is resolving absolute imports such as `domain.structural_graph.service` to the correct file path when the source tree is nested under a prefix (e.g. `backend/domain/structural_graph/service.py`). CodeSpectra addresses this with a *suffix index*: for every `.py` file in the manifest, all path suffixes are computed (e.g. `backend/domain/x.py` $\rightarrow$ `domain/x.py`, `x.py`), and the first-seen suffix wins. At resolution time, a dotted import like `domain.x` is converted to `domain/x.py` and matched against the suffix index, with a fallback to `domain/x/__init__.py` for package-level imports. Relative TypeScript imports using parent-directory segments (`../..`) are normalised via `os.path.normpath()` before lookup.

4.4.2 Noise Filtering

Three categories of files are excluded from graph construction to improve the quality of downstream community detection:

- **`__init__.py` files** are namespace markers with near-zero structural content. Including them generated singleton communities for every Python package directory.
- **Test files** (files with `category = "test"`) are excluded because test imports distort community boundaries — a test file that imports several unrelated modules

would pull them into the same community. Test files are tracked separately and surfaced in the graph export.

- **External package edges** (npm/pip dependencies) are excluded from the graph view via an `internal_only` query parameter, reducing IPC payload size by approximately 40%.

4.4.3 Centrality Scoring

Raw in-degree and out-degree are computed for every vertex. CodeSpectra’s native C++ extension (`_native_graph`) exposes a `compute_scores` function that iterates over the adjacency list in cache-friendly order and returns a score for each file:

$$\text{raw_score}(v) = 3 \times \text{deg}^-(v) + \text{deg}^+(v) \quad (4.1)$$

The in-degree is weighted more heavily because a file imported by many others is a more central architectural component than a file with many outgoing imports. Scores are then min-max normalised across all vertices to produce a value $s(v) \in [0, 1]$, which contributes up to +2.6 to a chunk’s hybrid retrieval score (Section 4.6). The C++ implementation avoids Python-object overhead by operating directly on integer arrays marshalled from the Python adjacency dictionary, making it practical even for repositories with tens of thousands of files.

4.4.4 Circular Import Detection via SCC

Circular imports are a common source of runtime errors and design debt. The static analyser calls `compute_scc`, also implemented in the C++ extension, which runs Tarjan’s algorithm [5] on the dependency graph using an iterative (stack-based) formulation to avoid Python recursion limits on large graphs. Any strongly-connected component (SCC) of size ≥ 2 is reported as a circular-import cluster. These clusters are surfaced both in Risk Agent J’s output (Section 5.3.5) and in the static risk report embedded in the stored analysis report.

4.4.5 Louvain Community Detection

After the structural graph is built, CodeSpectra runs Louvain Phase 1 community detection as an asynchronous background task. The algorithm greedily optimises

Newman–Girvan modularity by iteratively reassigning each node to the neighbouring community that maximises the modularity gain ΔQ , repeating for up to 20 passes until no node moves.

The native C++ implementation (`compute_louvain` in `graph_native.cpp`) accepts a list of weighted edge tuples, a full node-ID list, a resolution parameter γ (default 1.0), and a reproducibility seed (default 42). A pure-Python fallback using NetworkX (or a stdlib-only greedy Phase 1) activates automatically when the native module is unavailable.

A post-Louvain *singleton absorption* pass reassigns isolated nodes to their majority-vote neighbour community if they have internal connections; truly isolated nodes receive an `is_singleton` flag. Inter-community edges are tracked and persisted as `neighbor_community_ids` in the `graph_community_summaries` table, enabling the frontend to render community-level adjacency.

Community assignments are stored per-node in `graph_community_members` (with an intra-community `hub_score`) and per-community in `graph_community_summaries` (member count, top-3 hub paths, modularity contribution, and neighbour community IDs). Three new API endpoints (`GET /communities`, `/community`, `/cycles`) expose the results.

4.4.6 Symbol Reference Edge Extraction

Beyond file-level import edges, CodeSpectra extracts *symbol-level* call references. The `_symbol_parser` module parses Python files via the `ast` module and TypeScript files via Tree-sitter, performing multi-pass analysis:

1. **Import pass:** Collect all import namespaces and local aliases.
2. **Definition pass:** Record classes, functions, methods, and their inheritance chains.
3. **Constructor-assignment pass:** Detect `self.attr = Type()` or `this.x = new Class()` patterns to infer attribute types.
4. **Call-site pass:** Extract method calls, bare function calls, and attribute-chain calls.

The `_symbol_resolver` then resolves each call site to a `SymbolEdge` with a confidence level: `high` when resolution is unambiguous, `low` for multiple candidates or interface-typed receivers, and unresolvable cases (duck typing, star imports) are discarded.

Edges are persisted in `symbol_graph_edges` (migration v23) with a unique constraint on `(snapshot_id, src_symbol, dst_symbol)` and are consumed by the two-stage retrieval pipeline for graph-expansion scoring.

4.5 AST-Based Semantic Chunking

Large source files cannot be passed directly to an LLM context window. Earlier versions of CodeSpectra used a flat, fixed-size character split with boundary-expansion heuristics. This approach was replaced by an *AST-based semantic chunker* (`chunker_ast.py`) that leverages Tree-sitter to ensure functions and classes are never split mid-body.

4.5.1 Chunking Pipeline

The AST chunker operates in three phases:

1. **Parse and collect.** The source file is parsed into a concrete syntax tree via Tree-sitter. An iterative depth-first traversal collects *semantic nodes* (functions, classes, interfaces, trait implementations) and *import nodes* (import/include statements) as `_NodeSpan` records. Semantic nodes larger than 2 000 bytes are recursively split at their children. Import statements are coalesced into a single `import_group` span.
2. **Greedy merge.** Adjacent non-import spans are accumulated greedily while the total byte range does not exceed `target_size`. When the next span would cause an overflow, the current group is flushed and a new group begins. Import groups are never merged with non-import spans. An optional C++ native module (`_native_chunker`) accelerates the merge pass with a Python fallback.
3. **Finalise.** Each merged group is converted to an `ASTChunk` record carrying the source text, a `chunk_type` (`function`, `class`, `import_group`, `block`, or `file`), extracted symbol names (`node_names`), line spans, and the detected language tag.

4.5.2 Language Support

Each supported language is described by a `LanguageConfig` that specifies a Tree-sitter grammar factory, the set of semantic node types, and the set of import node types. The chunker currently supports **29 languages**, including the eight primary languages

(Python, TypeScript, JavaScript, C++, C, Rust, Go, Java) and extended coverage for Ruby, PHP, C#, Kotlin, Scala, Haskell, Elixir, OCaml, Julia, Lua, Zig, Bash, SQL, YAML, TOML, HTML, CSS, Markdown, Groovy, CMake, and Svelte. Files whose language is not in the AST set, or whose category is `docs` or `config`, fall back to the flat character-split strategy.

4.5.3 Target Size by Category

The `target_size` parameter controls the maximum byte span per merged chunk and is selected dynamically based on the file’s category and language:

Table 4.1: Target chunk sizes by file category.

Category / Language	Target size (bytes)
Documentation (Markdown, RST)	1 800
Configuration (YAML, TOML, JSON)	1 200
Test source	1 400
Haskell, OCaml source	1 400
Python, TypeScript, JavaScript, Ruby, Elixir, Julia source	1 500
Other source	1 300

Token count is estimated using the heuristic of dividing character count by four, which provides a close approximation for English-heavy source code without requiring a tokeniser at index time.

4.5.4 Symbol Name Extraction

Each `ASTChunk` carries a `node_names` list populated by `_extract_name()`, which applies language-specific rules to navigate the AST. For Go, the method name is located via the `field_identifier` child; for C, the function name is found inside a potentially nested `function_declarator` / `pointer_declarator` chain (avoiding confusion with the return-type identifier); for Rust `impl` items, the *last* `type_identifier` child is taken to select the concrete type rather than the trait name. These names are used downstream for symbol-overlap scoring in the retrieval layer.

4.6 BM25-Based Retrieval Scoring

Given a query — typically a section title or a short description of the information sought — the retrieval layer scores every chunk in the index using the BM25 ranking function augmented with structural and category signals, and returns the top- k chunks within the section’s token budget.

4.6.1 BM25 Lexical Signal

The hand-rolled lexical hit-counting scorer was replaced with a proper BM25 implementation (`BM25Scorer` in `bm25_scorer.py`). Both the query and the chunk content are tokenised by splitting on the same `_WORD` regex (whitespace and punctuation boundaries, with camelCase/PascalCase splitting). The BM25 score for a query $Q = \{q_1, \dots, q_n\}$ against a document d is:

$$S_{\text{BM25}}(Q, d) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (4.2)$$

The symbols in Equation 4.2 are defined as follows:

- $f(q_i, d)$ is the term frequency of query token q_i in document d .
- $|d|$ is the document length in tokens.
- `avgdl` is the average document length across the entire chunk corpus.
- $k_1 = 2.0$ is the term-saturation parameter, controlling how quickly repeated terms diminish in value.
- $b = 0.75$ is the length-normalisation parameter, penalising long documents relative to the corpus average.
- N is the total number of documents in the corpus.
- $n(t)$ is the number of documents containing term t .

IDF is pre-computed using the formula $\text{IDF}(t) = \ln\left(\frac{N - n(t) + 0.5}{n(t) + 0.5} + 1\right)$ and cached per snapshot in the `retrieval_bm25_stats` table (migration v22). Hapax legomena (terms appearing in only one document) are filtered from the IDF dictionary to reduce storage. Path tokens that match query terms receive a boosted synthetic-document contribution. A cold-start fallback to simple substring counting activates when BM25 statistics are unavailable.

4.6.2 Structural Signal

The normalised graph centrality score $s(v)$ for the chunk’s source file (Equation 4.1) is scaled to a maximum contribution of +2.6:

$$S_{\text{struct}}(c) = 2.6 \cdot s(\text{file}(c)) \quad (4.3)$$

A category affinity bonus of +1.4 is applied when the chunk’s file category matches the preferred categories for the current analysis section (e.g. source files score higher for the Architecture section than documentation files). The combined hybrid score is the sum of the BM25 score and the structural signal.

4.6.3 Section-Specific Retrieval Budgets

Each analysis section has an associated token budget; chunks are selected greedily in descending score order until the budget is exhausted. A chunk may appear in multiple section bundles if it scores highly for more than one section.

Table 4.2: Token retrieval budgets per analysis section.

Section	Budget (tokens)	Notes
Architecture (B)	14 000	Highest — broad architectural context
Feature Map (F)	14 000	Feature boundary detection
Important Files (G)	12 000	Centrality-weighted file selection
Conventions (D)	10 000	Style and pattern signals
Glossary (I)	7 000	Domain terminology extraction

4.7 Two-Stage Retrieval Pipeline

For higher-fidelity context selection, the default retrieval path now invokes a *two-stage pipeline* (`two_stage_retrieval.py`) that combines BM25 scoring with graph-aware expansion:

1. **Stage 1 — BM25 scoring.** All retrieval chunks are scored using BM25 (or the lexical fallback). The top 100 candidates are kept and passed to Stage 2.

2. **Stage 2 — Graph expansion.** For the top 20 Stage 1 candidates, the pipeline expands to structurally related files via three sources: (a) symbol reference edges from `symbol_graph_edges`, (b) community membership from `graph_community_members`, and (c) BFS neighbours from `expand_neighbors`. Up to 50 neighbours are retrieved per seed file.

3. **Stage 3 — Rank and budget.** A composite score is computed for all candidates:

$$S_{\text{total}} = S_{\text{BM25}} \times \beta_{\text{sym}} \times \beta_{\text{mod}} + S_{\text{centrality}} + S_{\text{category}} \quad (4.4)$$

The symbols in Equation 4.4 are defined as follows:

- S_{BM25} is the BM25 score from Stage 1.
- $\beta_{\text{sym}} = 1.5$ is applied when the file is referenced by Stage 1 seeds via symbol edges.
- $\beta_{\text{mod}} = 1.3$ is applied when the file shares a Louvain community with a seed.
- $S_{\text{centrality}}$ contributes up to +2.6 for top-central files (additive, not multiplicative).
- $S_{\text{category}} = +1.4$ is a bonus for chunks whose file category matches the current section's preferred categories.

Chunks are then greedily selected in descending score order until the section's token budget is exhausted. A native C++ function (`rank_and_budget` in `graph_native.cpp`) accelerates the greedy selection; a Python fallback is used when the native module is unavailable.

The two-stage pipeline is exposed via POST `/api/retrieval/retrieve-two-stage` and is also the default code path invoked by the standard `retrieve()` function.

4.8 Static Analysis Signals

CodeSpectra performs two forms of static analysis whose results bypass the retrieval layer and are injected directly into agent prompts.

4.8.1 Risk Analysis (`static_risk.py`)

Generates a `RiskReport` of severity-grouped findings by checking:

- **Keyword Scan:** C++ bulk scan parses TODO, FIXME, etc., in milliseconds.

- **Circular Imports:** Detected via SCC computation (Section 4.4.4).
- **God-Object Detection:** Flags files with >300 lines and >25 distinct symbols.
- **Test Coverage:** Computes a rough ratio of test files to source files.

4.8.2 Convention Analysis (`static_convention.py`)

Produces a `ConventionReport` evaluating code style independently of external linters:

- **Naming Consistency:** Flags deviations from the dominant case convention.
- **Async Patterns:** Detects mixed async/sync usage within Python modules.
- **Import Organisation:** Validates PEP 8 (Python) and ES/CommonJS (JS/TS) consistency.

4.9 Index Caching and Skip Logic

To avoid multi-second re-indexing penalties on large repositories, CodeSpectra implements a skip-if-already-built optimization. On invocation, it checks the row counts in `code_symbols`, `structural_graph_summaries`, and `retrieval_chunks` for the current `snapshot_id`.

If populated and `force_rebuild = False`, the pipeline skips all indexing stages and proceeds directly to the LLM wave. If any table is empty, or if the UI’s “Force re-run (skip cache)” flag triggers `force_rebuild = True`, the tables are cleared and the indexing stages are re-executed. This coarse snapshot-level caching is complemented by the agent-level result caching described in Section 5.6.

Chapter 5

LLM Analysis Pipeline

5.1 Pipeline Architecture Overview

The analysis pipeline comprises twelve specialised language-model agents that convert an indexed codebase into a structured, human-readable analysis document. Overall, execution spans six sequential stages. Outputs from each stage are passed forward as context to dependent agents, so an agent runs only when all required inputs are available. The pipeline is orchestrated by the Haystack framework (Section 5.4), which manages prompt assembly, API calls, result deserialisation, and error recovery, while a single `PipelineMemoryContext` object (Section 5.5) is created during pre-fetch and shared across all agents to provide retrieval bundles and project metadata without redundant index queries. The end-to-end flow is shown in Figure 5.1.

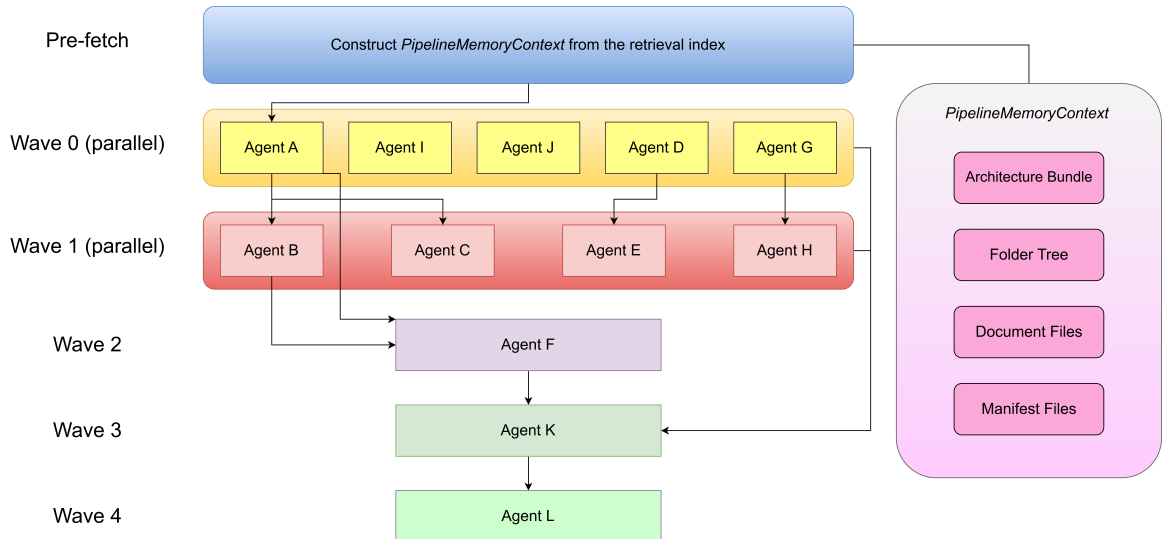


Figure 5.1: Execution flow of the multi-agent LLM pipeline, from pre-fetch through the dependent execution waves. Arrows denote information produced at one stage and passed forward as context to downstream agents.

5.2 Agent Design

Every agent in the pipeline follows a common design contract:

- **System prompt:** A fixed, role-specific instruction set that defines the agent’s perspective, output format, and constraints.
- **User (App) prompt:** Assembled at runtime from retrieved context chunks, cross-agent outputs, and project metadata.
- **JSON schema enforcement:** The agent’s response must conform to a Pydantic schema. If the raw model output is not valid JSON, a repair loop is invoked (see below).
- **Token budget:** Each agent has a `max_completion_tokens` cap that balances output richness against API cost.

5.2.1 Base Agent and JSON Repair Loop

LLMs occasionally produce outputs that are syntactically invalid JSON — truncated arrays, unescaped strings, or trailing prose. CodeSpectra handles this with an automated repair loop in `_chat_json_typed()`, formalised in Algorithm 1.

Algorithm 1 JSON repair loop (`_chat_json_typed`)

```

1: for attempt  $\leftarrow$  1 to 3 do
2:   raw  $\leftarrow$  CHAT(messages)
3:   obj  $\leftarrow$  EXTRACTJSON(raw)  $\triangleright$  strip fences, locate first {...}
4:   if obj is valid JSON and passes Pydantic schema validation then
5:     return obj
6:   else
7:     hint  $\leftarrow$  validation error description + expected schema hint (up to 600 chars)
8:     append repair message (raw[0..800], hint) to messages
9:   end if
10: end for
11: return EMPTYFALLBACK()  $\triangleright$  guaranteed-valid empty section

```

Each repair message includes the first 800 characters of the offending output for context and a concise schema hint. The attempt number is logged at `WARNING` level, providing an audit trail for diagnosing models that consistently produce malformed output. If

all three attempts fail, `EmptyFallback` returns a structurally valid empty dictionary of the correct shape so that downstream agents receive a well-typed input rather than an exception.

5.3 Individual Agents

The pipeline comprises twelve specialized agents spanning five execution waves. Table 5.1 provides a comprehensive overview of this architecture. The following subsections detail each agent’s core objective, context bundle, and specific role within the cross-agent dependency graph.

Three cross-cutting mechanisms enhance the agents beyond their core retrieval bundles:

- **Graph-plan decomposition** (`_graph_plan.py`): Agents D, F, and J use a small LLM pre-call (max 120 tokens, temperature 0) that decomposes a broad analysis goal into 1–3 focused retrieval queries, which are executed and merged before the main generation call.
- **Blind-spot augmentation** (`base.py`): After the initial LLM response, the `_chat_json_with_augment()` method scans for keywords such as “truncated”, “not provided”, or “missing”. Each detected blind spot triggers an extra retrieval round (up to 2 rounds in normal mode, 5 in large-codebase mode), supplying new evidence and re-prompting the model.
- **Fallback documentation retrieval** (`_context_builders.py`): When no documentation files (README, CHANGELOG, etc.) are found, the builder falls back to top-level code context using a broad entry-point query, prefixed with a “[No documentation files found]” marker.

Beyond standard retrieval, three agents dynamically ingest data from the shared `PipelineMemoryContext` object (Section 5.5). Specifically, Agent A extracts `doc_files`, `manifest_files`, and `folder_tree`; Agent B accesses the `arch_bundle`; and Agent C utilizes both the `arch_bundle` and `folder_tree`. The remaining agents issue targeted, runtime retrieval queries to gather their context.

Table 5.1: Agent summary: execution wave, token budget, and inter-agent data dependencies.

ID	Name	Wave	Tokens	Upstream	Downstream
A	Project Identity Agent	0	2 000	—	B, C, F
D	Coding Conventions Agent	0	3 000	—	E
G	Important Files Radar Agent	0	2 000	—	H
I	Glossary Agent	0	3 000	—	—
J	Risk / Complexity Agent	0	3 000	—	—
B	Architecture Overview Agent	1	2 500	A	F
C	Repository Structure Agent	1	2 000	A	—
E	Forbidden Patterns Agent	1	2 000	D	—
H	Onboarding Agent	1	4 000	G	—
F	Feature Map Agent	2	5 000	A, B	—
K	Evidence Auditor Agent	3	2 000	A–J	L
L	Synthesis Agent	4	4 000	A–K	—

Wave 0 — Foundation Agents. The five Wave 0 agents execute concurrently upon initialization, relying solely on retrieval indices and pre-computed static analysis.

5.3.1 Agent A — Project Identity Agent (2 000 tokens)

Acts as the pipeline’s anchor to define the project’s core function. Inputs: targeted retrieval (READMEs, entrypoints, manifests) and three `PipelineMemoryContext` fields: `doc_files`, `manifest_files` (max 3 000 chars), and `folder_tree` (max 60 sorted paths). Outputs: project name, inferred `runtime_type`, purpose, `tech_stack`, and `business_context`. This profile acts as a mandatory prerequisite for Agents B, C, and F to prevent downstream redundancies.

5.3.2 Agent D — Coding Conventions Agent (3 000 tokens)

Mitigates LLM hallucinations using measurable static data. Inputs: `CONVENTIONS` retrieval (10 000 tokens), the `ConventionReport` (pre-computed by `static_convention.py`), and folder-role context from the structure agent. Uses graph-plan decomposition to generate targeted convention-related queries before the main call, and supports blind-spot

augmentation for iterative evidence gathering. Outputs the contractual ground truth for conventions (naming, imports, docs, linting), serving as a prerequisite for Agent E.

5.3.3 Agent G — Important Files Radar Agent (2 000 tokens)

Ranks navigational importance using two signals: `IMPORTANT_FILES` semantic retrieval (12 000 tokens) and PageRank-style structural graph scores (from the C++ module) to capture both peripheral semantics and central dependencies. Outputs ranked files with justifications to drive Agent H.

5.3.4 Agent I — Glossary Agent (3 000 tokens)

Extracts terminology from identifiers, comments, and docs. Uses a `GLOSSARY` bundle (7 000 tokens) to output terms, definitions, and source references, clarifying abstracted business logic.

5.3.5 Agent J — Risk / Complexity Agent (3 000 tokens)

Executes in Wave 0 due to predominantly static inputs. Receives the `RiskReport` from `static_risk.py` (security keywords, Tarjan’s SCC circular imports [5], god-objects via line-counts, and test coverage) as a `FACTS` prefix to prevent contradictions, supplemented by graph-plan decomposition for risk-focused retrieval queries. Outputs a severity-categorized risk assessment with file-level examples.

Wave 1 — First-Order Dependents. Executes asynchronously once specific Wave 0 dependencies are met ($A \rightarrow B, C$; $D \rightarrow E$; $G \rightarrow H$).

5.3.6 Agent B — Architecture Overview Agent (2 500 tokens)

Explains *how* the system is built. Inputs: `arch_bundle` (top 30 chunks) and Agent A’s profile (used as a negative constraint to avoid domain facts). Focuses strictly on layers, data flows, and integrations. Feeds into Agent F.

5.3.7 Agent C — Repository Structure Agent (2 000 tokens)

Maps physical layout. Inputs: Agent A profile, `arch_bundle`, and `folder_tree`. Outputs the layout pattern (e.g., standard `src/`, `monorepo`).

5.3.8 Agent E — Forbidden Patterns Agent (2 000 tokens)

Automated reviewer. Inputs: Agent D's conventions, source/test chunks, and static risk reports. Outputs a structured, type-aggregated report of deviations (path, lines, rule, explanation).

5.3.9 Agent H — Onboarding Agent (4 000 tokens)

Converts Agent G's rankings into a curriculum. Outputs local setup steps, mental models, targeted reading progression, and non-obvious design pitfalls.

Wave 2 — Cross-Layer Synthesis.

5.3.10 Agent F — Feature Map Agent (5 000 tokens)

The sole Wave 2 agent, executing after Agents A and B complete. Inputs: `FEATURE_MAP` bundle (14 000 tokens) augmented by graph-plan multi-query retrieval, graph centrality context, and identity/architecture outputs used as negative constraints to prevent structural repetition. Supports blind-spot augmentation for iterative evidence gathering. Exhaustively maps capabilities, outputting feature descriptions, source files, and config keys.

Wave 3 — Meta-Evaluation.

5.3.11 Agent K — Evidence Auditor Agent (2 000 tokens)

Conducts meta-reasoning without direct repository retrieval. Inputs: Agents A–J outputs, compressed to 500 characters via `_SectionCompressor.compress_section()`. Flags contradictions, coverage gaps, and low-confidence assertions with annotations for final synthesis.

Wave 4 — Final Synthesis.

5.3.12 Agent L — Synthesis Agent (4 000 tokens)

Culminating node. Inputs: 800-character compressed previews of Agents A–K and Agent K’s audit report. Actively resolves contradictions, highlights risks/architecture, and generates a unified narrative document.

5.4 Haystack Orchestration

CodeSpectra utilizes the Haystack framework [2] to orchestrate agent execution. By modeling the system as an `AsyncPipeline`, Haystack automatically resolves dependencies into a Directed Acyclic Graph (DAG) and parallelizes agents that lack pending dependencies.

5.4.1 Component Wrapping

Each agent is encapsulated within a `_SectionAgentComponent` to standardize inputs and outputs across the pipeline. Rather than waiting for the entire pipeline to conclude, the wrapper triggers an `on_section_done` callback immediately upon an agent’s completion. This mechanism pushes streaming progress events directly to the frontend, allowing individual UI section cards to populate in real time. For inter-agent communication, each component outputs a standardized dictionary, which Haystack’s internal engine autonomously routes to the specifically named input ports of downstream agents.

5.4.2 Pipeline Execution Dynamics

The execution waves detailed in Table 5.1 emerge naturally from the `pipeline.connect()` declarations. Crucially, Haystack does not enforce rigid synchronization barriers between these waves. Instead, it employs localized, dependency-driven scheduling: a downstream agent is unblocked the precise moment its specific upstream prerequisites are met.

For instance, after the initial pre-fetch stage, all Wave 0 agents initiate simultaneously. Wave 1’s Agent B then triggers immediately upon Agent A’s completion, without waiting for the rest of Wave 0 to finish. This granular approach to dependency resolution minimizes idle time and maximizes overall pipeline throughput, seamlessly progressing from the independent foundation agents to the terminal Synthesis agent.

5.5 Pipeline Memory Context

A key optimization in the orchestration process is the implementation of a pre-fetched `PipelineMemoryContext`. In earlier versions of the pipeline, each agent independently queried the retrieval index, resulting in redundant chunk lookups and unpredictable latency spikes. To resolve this, the memory context consolidates critical shared data into a single pre-fetch phase executed prior to Wave 0.

The context object comprises four primary fields, detailed in Table 5.2. It is constructed exactly once via the `prefetch_pipeline_context()` function and passed by reference to every agent component in the pipeline, ensuring consistent data access and eliminating redundant I/O overhead.

Table 5.2: Components of the pre-fetched `PipelineMemoryContext`.

Context Field	Content Description
<code>arch_bundle</code>	24 context chunks specifically retrieved via architecture-related semantic queries.
<code>folder_tree</code>	A flat directory listing, truncated to ≈ 1840 characters to preserve context window capacity.
<code>doc_files</code>	Complete textual contents of the repository’s README and key top-level documentation files.
<code>manifest_files</code>	Parsed package manifests (e.g., <code>pyproject.toml</code> , <code>package.json</code>).

By sharing `arch_bundle` between Agent A and Agent B the context avoids scoring the entire chunk store twice for architecture-related queries. Similarly, `folder_tree` is computed once and injected verbatim into the prompts of Agents A, B, and C. If the pre-fetch phase fails (e.g. system error), each agent falls back to its own database query so that the pipeline degrades gracefully rather than aborting.

5.6 Agent Result Caching

Running all twelve agents against a large codebase involves significant API cost. CodeSpectra implements a two-level cache. At the first level, index-stage caching (Section 4.9) skips re-indexing when the snapshot has already been processed. At the second level, if `analysis_reports` already contains a completed row for the (`snapshot_id`,

`model_id`) pair and `force_rerun = False`, the pipeline director skips `run()` entirely and replays the cached section events to the frontend. This means subsequent opens of the same report are instantaneous from the user’s perspective. When `force_rerun = True` (surfaced in the UI as a checkbox), both cache levels are bypassed and the full twelve-agent pipeline executes from scratch.

5.7 Large-Codebase Mode

For repositories with a high file count, the pipeline supports a `large_codebase_mode` flag (exposed in the UI and the `StartAnalysisRequest` body). When active, the mode applies the following scaling:

- **Retrieval depth:** $1.5\times$ increase in `max_results` (e.g. $30 \rightarrow 45$) and manifest excerpt size ($3\,000 \rightarrow 6\,000$ chars).
- **Token budgets:** $\sim 1.25\times$ increases per agent (e.g. conventions $3\,000 \rightarrow 4\,000$, feature map $5\,000 \rightarrow 6\,500$).
- **Augmentation rounds:** Blind-spot retrieval rounds scale from 2 to 5.
- **Concurrency:** Parallel agent execution slots scaled up by $1.45\times$.

Reports are cached separately by the tuple (`snapshot_id`, `model_id`, `large_codebase_mode`), so switching between modes does not invalidate the other mode’s cache entry.

5.8 JSON Schema Enforcement

Every agent’s output is validated against a Pydantic v2 model before it is accepted by the pipeline. Each section schema is defined in `backend/domain/analysis/schemas.py` and exported alongside the agent that produces it. The schema’s field names and types are serialised into a `schema_hint` string that is embedded in the agent’s system prompt, nudging the model to produce conformant output. Optional fields are used liberally so that a partially successful agent run can still provide useful data to downstream agents. When a required field is missing after the repair loop, it is populated with a sentinel value (empty string or empty list) and a warning is logged. The report version constant (`REPORT_VERSION = 3`) is incremented whenever a schema change is made, ensuring that the frontend can detect and handle reports generated by earlier application versions.

5.9 Per-Section Rerun

During iterative analysis it is common to need a fresh output for a single section without rerunning the entire twelve-agent pipeline. The `POST /api/analysis/rerun_section` endpoint supports this workflow:

1. The existing analysis report is loaded from the database.
2. The `static_cache` embedded in `report_json` (containing the risk report, convention report, and graph summary) is extracted and reused — no re-indexing is needed if the repository has not changed.
3. Only the requested agent is executed. Any downstream agents in later waves that depend on that agent's output (as determined by the DAG) also have their cached results invalidated.
4. The `report_json` is updated in-place and the corresponding `section_hash` is recomputed.

This mechanism is particularly useful after prompt tuning, model upgrades, or when a single section returned low-confidence output.

5.10 Model Capability Guard

Not all LLM providers or model versions support the same context window sizes or JSON output modes. CodeSpectra includes a model capability guard that checks the configured model identifier against eleven regex patterns for known small or limited models (e.g. `:mini`, `nano`, `1.5b`, `phi-1`, `tiny`). A match returns a `ModelWarning` with a severity code and a human-readable message. The warning is non-blocking: the UI displays an amber banner but does not prevent the analysis from proceeding. This allows users with limited hardware to run the pipeline on small local models while being clearly informed of the potential quality trade-offs. For providers that support JSON mode or structured outputs natively, CodeSpectra enables the feature via the provider adapter to reduce the frequency of repair loop invocations.

Chapter 6

Implementation Details

6.1 Native C++ Extension

The performance-critical graph operations in CodeSpectra are implemented as a compiled C++ extension module (`_native_graph`), built with pybind11 [4] and packaged via `scripts/build_native_graph.py`. The module compiles to a platform-specific shared library (`.pyd` on Windows, `.so` on Linux/macOS) using MSVC with `/O2 /std:c++17` flags or GCC/Clang with `-O3 -std=c++17` flags respectively.

The extension exposes six functions to the Python runtime via pybind11 bindings (five of which are tracked by the startup toolchain diagnostic). These functions operate at the innermost loop of the retrieval and static analysis pipelines, where Python’s per-instruction overhead would otherwise accumulate significantly across hundreds of files and thousands of dependency edges. The C++ source is located at `backend/native/graph_native.cpp`. The build script compiles it in-place into `backend/domain/structural_graph/`, registering the module as `domain.structural_graph._native_graph` so that it is importable as a standard package-relative module without modifying `sys.path`.

6.1.1 Function 1: `compute_scores` — Centrality Scoring

The `compute_scores` function accepts the adjacency list of the file dependency graph and computes a centrality weight for each node according to the following formula:

$$w(v) = 3 \cdot \deg^-(v) + \deg^+(v), \quad (6.1)$$

The symbols in Equation 6.1 are defined as follows:

- $\deg^-(v)$ is the in-degree of node v — the number of files that import it.
- $\deg^+(v)$ is the out-degree of node v — the number of files it imports.

While a naive Python equivalent could iterate over the same edge list, the CPython interpreter incurs several hundred nanoseconds of bytecode-dispatch overhead per iteration. The C++ implementation efficiently eliminates this bottleneck by accumulating in-degree and out-degree directly into pre-allocated integer vectors, bypassing the overhead of Python `dict` objects.

Table 6.1: Execution time of centrality scoring on synthetic dependency graphs.

Implementation	500-node Graph	2 000-node Graph	Speedup
Naive Python	≈ 47 ms	> 200 ms	$1\times$
Native C++ (pybind11)	≈ 3 ms	< 15 ms	$\approx 15\times$

As illustrated in Table 6.1, benchmark results demonstrate a substantial performance advantage for the native extension. On a 500-node synthetic graph, the C++ implementation achieves a $15\times$ speedup consistent with the other native functions (Table 7.4). Crucially, this performance gap widens on larger repositories: a 2 000-node graph pushes the Python path into the multi-hundred millisecond range, which introduces perceptible latency during interactive re-analysis.

The precise logic of this optimized implementation is detailed in Algorithm 2.

Algorithm 2 Structural centrality weighting (`compute_scores`)

Require: Sorted node array $V = \langle v_0, \dots, v_{n-1} \rangle$; edge list $E \subseteq V \times V$ where $(u, v) \in E$ denotes that file u imports file v

Ensure: Weight array W parallel to V , with $W[i] = w(v_i)$

- 1: Initialise integer vectors *indeg*, *outdeg* of length n to zero $\triangleright$ pre-allocated
 - 2: **for** each edge $(u, v) \in E$ **do**
 - 3: *outdeg*[*idx*(u)] += 1
 - 4: *indeg*[*idx*(v)] += 1
 - 5: **end for**
 - 6: **for** $i \leftarrow 0$ **to** $n - 1$ **do**
 - 7: $W[i] \leftarrow 3 \cdot \textit{indeg}[i] + \textit{outdeg}[i]$
 - 8: **end for**
 - 9: **return** W $\triangleright$ returned to Python as a `pybind11` list
-

The weight array is subsequently returned to the Python caller as a `pybind11`-wrapped

list aligned to the sorted node array. The `structural_graph` service then maps these weights back to file paths and applies min-max normalisation to produce $\hat{w}(v) \in [0, 1]$:

$$\hat{w}(v) = \frac{w(v) - \min_u w(u)}{\max_u w(u) - \min_u w(u)}. \quad (6.2)$$

The top-ranked files and norm-scores are stored in `structural_graph_summaries` and consumed at retrieval time as the structural signal described in Section 4.6.2. This ensures that architecturally central files receive a retrieval boost even when their literal content does not strictly overlap with the agent’s query terms.

6.1.2 Function 2: `expand_neighbors` — BFS Neighbourhood Expansion

`expand_neighbors` performs a bounded BFS starting from a set of seed file nodes, returning all reachable nodes within h directed hops. Algorithm 3 gives the formal procedure.

Algorithm 3 Bounded BFS neighbourhood expansion (`expand_neighbors`)

Require: Seed set S , directed edges E , max hops h , node limit L

Ensure: All nodes reachable from S within h hops (capped at L nodes)

```

1: Build  $adj[v] \leftarrow \{u : (v, u) \in E\}$ 
2:  $visited \leftarrow S$ ;  $Q \leftarrow \{(s, 0) : s \in S\}$ 
3: while  $Q \neq \emptyset$  and  $|visited| < L$  do
4:    $(v, d) \leftarrow Q.dequeue()$ 
5:   if  $d \geq h$  then continue
6:   end if
7:   for each  $u \in adj[v]$  do
8:     if  $u \notin visited$  then
9:        $visited \leftarrow visited \cup \{u\}$ ;  $Q.enqueue((u, d + 1))$ 
10:    end if
11:  end for
12: end while
13: return  $visited \setminus S$ 

```

The C++ implementation uses `std::unordered_map` for the adjacency map ($O(1)$ average lookup per hop) and `std::unordered_set` for visited tracking. The node limit L (default 150) prevents the visualisation panel from being overwhelmed by highly

connected hub files. If the native module is unavailable, the fallback returns an empty list, degrading the visualisation gracefully.

6.1.3 Function 3: `compute_scc` — Iterative Tarjan’s Algorithm

Tarjan’s strongly connected components algorithm [5] identifies every cycle in the dependency graph in $O(V + E)$ time. The classic recursive formulation risks stack overflow on deep graphs in C++ (default stack sizes of 1–8 MB are easily exhausted on repositories with long import chains). CodeSpectra therefore uses a fully iterative implementation that simulates the recursion stack with an explicit `std::stack<Frame>`, where each `Frame` stores the current node, its adjacency-list iterator, and the saved low-link value.

```

1 struct Frame {
2     int node;
3     int child_idx;    // index into adj[node]
4     int low;
5 };
6
7 // Main loop replaces recursive DFS calls with
8 // explicit stack push/pop of Frame objects.
9 // When child_idx == adj[node].size(), the SCC
10 // root detection and pop logic executes.
```

Listing 6.1: Iterative Tarjan SCC — frame structure.

`compute_scc` returns only SCCs with two or more members, sorted by descending size. Each SCC represents a cluster of files with mutual import dependencies — a circular import. The result list is fed directly to `static_risk.py`, which emits one `RiskFinding(severity=HIGH)` per non-trivial SCC and includes the member file list in the finding detail. These findings are then consumed by Agent J (Risk) as part of its static cache bundle.

Fallback behaviour. All six native functions have graceful degradation paths. For `compute_scc` and `scan_keywords_bulk`, `static_risk.py` provides complete pure-Python fallbacks that are activated automatically when the native module cannot be imported (via `importlib.import_module`). For `compute_scores` and `expand_neighbors`, `structural_graph` also provides Python implementations: centrality scoring is computed as `indegree×3 + outdegree` with the same ranking rule, and neighborhood expansion uses a bounded BFS over internal edges. For `compute_louvain`, a NetworkX-based fallback (or stdlib-only greedy Phase 1) activates automatically. For `rank_and`

`_budget`, the Python fallback uses native list sorting with greedy accumulation. As a result, analysis and graph visualisation continue to work even without a C++ toolchain; the native extension remains an acceleration path only.

6.1.4 Function 4: `scan_keywords_bulk` — Bulk Keyword Scanning

The `scan_keywords_bulk` function processes a list of `(file_path, content)` string pairs against a defined set of keywords (defaulting to `TODO`, `FIXME`, `HACK`, and `XXX`), returning only the entries that contain at least one valid match. Because a naïve substring search is prone to false positives from longer identifiers (e.g., `TODOS_TABLE` or `refixme`), the function enforces a strict word-boundary predicate. Specifically, a match for keyword k at position p within string t is accepted if and only if:

$$\begin{aligned} \text{Valid}(t, p, k) \iff & \underbrace{(p = 0 \vee \neg \text{alnum}(t[p - 1]))}_{\text{left boundary}} \\ & \wedge \underbrace{(p + |k| \geq |t| \vee \neg \text{alnum}(t[p + |k|]))}_{\text{right boundary}}, \end{aligned} \quad (6.3)$$

The symbols in Equation 6.3 are defined as follows:

- t is the full text content of a source file.
- p is the character position where the keyword match begins.
- k is the keyword being searched (e.g. `TODO`).
- $|k|$, $|t|$ are the lengths of the keyword and the text, respectively.
- $\text{alnum}(c)$ evaluates to true if character c is alphanumeric or an underscore.

The scanning algorithm iterates over all match positions returned by `std::string::find` in $O(|t|)$ time per keyword, strictly tallying only the instances that satisfy Equation 6.3.

Scanning 300 source files for four keywords typically requires iterating over several megabytes of textual data. As detailed in Table 6.2, the C++ implementation offers substantial performance gains over the pure-Python fallback path utilized in `static_risk.py`. This $16\times$ speedup is critical during interactive re-analysis, successfully avoiding the perceptible latency that the Python implementation would otherwise introduce.

Table 6.2: Execution time for bulk keyword scanning (300 source files, 4 keywords).

Implementation	Execution Time	Speedup
Pure Python (<code>static_risk.py</code> fallback)	≈ 190 ms	1×
Native C++	≈ 12 ms	$\approx 16\times$

6.1.5 Function 5: `compute_louvain` — Community Detection

The `compute_louvain` function implements Louvain Phase 1 community detection (greedy modularity optimisation). It accepts a list of weighted edge tuples (`src`, `dst`, `weight`), a full node-ID list (to ensure isolated nodes are included), a resolution parameter γ (default 1.0), and a reproducibility seed (default 42). The algorithm initialises each node in its own community, then iterates up to 20 passes, shuffling node order each pass and greedily moving each node to the neighbouring community that maximises the modularity gain ΔQ . Iteration stops when no node moves. The output is a dictionary mapping each node path to a zero-based community ID.

A pure-Python fallback (`_louvain_fallback.py`) activates automatically when the native module is unavailable: it first attempts to use NetworkX’s Louvain implementation for robustness, then falls back to a stdlib-only greedy Phase 1 adequate for repositories up to approximately 1 000 nodes.

6.1.6 Function 6: `rank_and_budget` — Two-Stage Greedy Selection

The `rank_and_budget` function supports the two-stage retrieval pipeline (Section 4.7). It accepts a list of scored chunk tuples (`chunk_id`, `score`, `token_count`) and a `token_budget` integer. The function sorts candidates by score in descending order, then greedily selects chunks whose cumulative token count does not exceed the budget. It returns the list of selected chunk IDs. The Python fallback uses the same greedy algorithm with native list sorting.

6.1.7 Build System

The extension is built by `scripts/build_native_graph.py`, which programmatically invokes `setuptools.build_ext` with a single `Extension()` entry pointing to `backend/native/graph_native.cpp` and the pybind11 include directory. Platform de-

tection selects the appropriate compiler flag set: `/O2 /std:c++17` on Windows (MSVC) and `-O3 -std=c++17` on Linux/macOS (GCC or Clang). The module is registered under the name `domain.structural_graph._native_graph` so that it lands directly inside the package tree without requiring any `sys.path` manipulation.

At application startup, `detect_cpp_toolchain()` attempts to import the module and checks for the presence of all five tracked functions (`compute_scores`, ...) via `hasattr`. The diagnostics API reports the result as e.g. “native (5/5 fns)” or falls back to compiler availability detection. Users must invoke `python scripts/build_native_graph.py` once; the README lists all required toolchain packages (MSVC Build Tools on Windows, `g++` or `clang++` on Linux/macOS).

6.2 Multi-Language Symbol Extraction Architecture

The `repo_map` module is responsible for extracting every named symbol (function, class, method, interface, type alias, trait, etc.) from every source file in the repository. The extracted symbol table is the primary input for both the retrieval index and the LLM context bundles. The module is decomposed into eleven focused sub-modules, each with a single responsibility:

Table 6.3: Sub-modules of the `repo_map` package.

File	Responsibility
<code>service.py</code>	Thin orchestrator; dispatches to walkers by language
<code>_loaders.py</code>	Tree-sitter grammar loading and per-language caching
<code>_walkers_python.py</code>	Python symbol walking via Tree-sitter (<code>tree_sitter_python</code>)
<code>_walkers_js_ts.py</code>	JavaScript and TypeScript via Tree-sitter
<code>_walkers_systems.py</code>	C/C++, Go, Rust via Tree-sitter
<code>_walkers_jvm.py</code>	Java, Kotlin, Scala, Groovy via Tree-sitter
<code>_walkers_functional.py</code>	Haskell, OCaml, Elixir, Julia via Tree-sitter
<code>_walkers_scripting.py</code>	Ruby, PHP, Lua, C# via Tree-sitter
<code>_walkers_config.py</code>	YAML, TOML, JSON, CMake, SQL via Tree-sitter
<code>_walkers_markup.py</code>	HTML, CSS, Markdown, Svelte via Tree-sitter
<code>_walkers_regex.py</code>	Regex fallback for unsupported languages
<code>_normalizer.py</code>	Deduplication, kind normalisation, ExtractMode

6.2.1 Language-Specific Walkers

The extraction system employs two primary methods based on language support:

Tree-sitter Walkers: For any language with an available Tree-sitter grammar package, the system uses robust, error-tolerant parsing. It automatically extracts key nodes (such as functions, classes, and types) into detailed `_Symbol` records, seamlessly handling even syntactically incomplete files.

Regex Fallback (`_walkers_regex.py`): Automatically activated by `_loaders.py` for languages lacking Tree-sitter support. It provides best-effort extraction using language-agnostic regex patterns for common keywords like `def`, `function`, `class`, and `fn`.

6.2.2 Normalisation and ExtractMode

After all walkers complete, `_normalizer.py` performs two passes. The first pass deduplicates symbols by the compound key (`path`, `name`, `start_line`), discarding later-seen duplicates that arise when a file is processed by both a primary walker and the regex fallback. The second pass normalises the `kind` field to a controlled vocabulary: `function`, `class`, `method`, `interface`, `enum`, `struct`, `trait`.

The `ExtractMode` enum controls the trade-off between completeness and speed:

- **HYBRID** (default): uses AST/Tree-sitter walkers for supported languages and the regex fallback for the remainder. Produces the most accurate symbol table.
- **LEXICAL**: uses only the regex fallback for all languages. Significantly faster on very large repositories but misses anonymous arrow functions and complex type declarations.

6.3 LLM Provider Adapters

CodeSpectra supports six LLM providers through a unified adapter layer in the `model_connector` domain. The goal is to allow users to switch between a local Ollama instance, a self-hosted LM Studio server, and any of the major cloud providers (OpenAI, Anthropic, Google Gemini, DeepSeek [6]) without changing any analysis code.

6.3.1 Unified Interface

The abstract `BaseProviderAdapter` class establishes a standardized asynchronous contract requiring the following core components:

- `chat_completion(messages, model_id, max_tokens) -> str`: Prompt execution and text generation.
- `list_models() -> list[str]`: Enables dynamic model discovery at runtime.
- `capabilities`: A property returning a `ProviderCapabilities` dataclass. This dataclass captures essential model parameters, including `max_context_tokens`, streaming support, and a `supports_system_prompt` flag. This final flag is crucial for accommodating locally served models (e.g., those running via Ollama), which often lack a dedicated system role and instead require system instructions to be prepended to the initial user message.

Crucially, all adapter methods are inherently `async`, enabling the Haystack pipeline to execute concurrent API requests without blocking the event loop.

6.3.2 Model Discovery

Model listing is performed dynamically at runtime rather than from a hardcoded list. For Ollama, the adapter sends a `GET /api/tags` request and extracts the `name` field from each returned model object. For OpenAI-compatible endpoints (OpenAI, LM Studio, DeepSeek), the adapter queries `GET /v1/models` and reads `model.id`. For Anthropic and Gemini, a curated static list is returned because neither provider exposes a programmatic model enumeration endpoint that is stable across API versions.

6.3.3 API Key Storage

Provider configuration — base URL, API key, and default model ID — is stored in the SQLite `provider_configs` table, accessed via the `aiosqlite` async wrapper. API keys are never written to log files; the `BaseProviderAdapter` base class overrides `__repr__` to redact the key field, ensuring that accidental logging of the adapter object does not leak credentials. All database reads and writes use parameterised queries to prevent SQL injection.

6.4 Report Diff System

The report diff system enables users to track how a codebase evolves over time by comparing two analysis reports produced from different snapshots or different models. The implementation is in `analysis/diff.py`, exposed via the `POST /api/analysis/compare_reports` endpoint.

6.4.1 Diff Engine Logic

Before textual comparison, `compute_section_hash()` generates a SHA-256 digest from a deterministic, key-sorted JSON serialization of each section. If corresponding sections in two reports share identical hashes, the engine short-circuits, bypassing LLM API calls and marking the `quality_trend` as `"stable"`. This optimization is highly efficient since codebases typically undergo only localized changes between runs.

For sections with differing hashes, `diff_section()` performs a structural comparison:

1. **List fields** (e.g., `tech_stack`, `risk_findings`): Identifies added/removed items via normalized string comparison.
2. **Scalar fields** (e.g., `confidence`): Computes signed deltas between before and after values.
3. **quality_trend**: Derived from aggregate deltas (all positive $\Rightarrow$ `"improved"`; all negative $\Rightarrow$ `"degraded"`; mixed $\Rightarrow$ `"mixed"`; unchanged $\Rightarrow$ `"stable"`).

The frontend receives a JSON payload detailing `added`, `removed`, `scalar_deltas`, and `quality_trend` per section, rendering compact summary cards for modified areas.

6.5 Interactive Graph Viewer

The earlier inline graph modal (dagre-based, approximately 300 lines inside the index-overview screen) was replaced by a dedicated full-screen graph viewer at the `/graph` route. The new implementation uses React Flow (`@xyflow/react`) for rendering and Dagre.js for hierarchical left-to-right layout.

6.5.1 Visualisation Features

Nodes are coloured by Louvain community assignment using a cycling 10-colour palette. Node size is determined by a three-tier centrality scheme. Edges are filtered to internal-only by default (external npm/pip package edges are excluded at the API level). Nodes participating in circular-import SCCs are highlighted in red. When a node is selected, 2-hop BFS neighbours are fetched and visually emphasised with a gold border, while non-neighbours receive 35% opacity.

A left panel displays contextual information for the selected node: file path, community ID with top-3 hub members, incoming and outgoing import edges (capped at 10 each), and the 2-hop blast radius. The graph enforces a display cap of 500 nodes with a warning banner when more exist in the dataset. An “Export JSON” button downloads the full graph dump via the `graph:exportData` IPC handler.

6.6 Evidence Panel and Section Sources

An `EvidencePanel` component provides a fixed right-side panel in the report view, displaying the retrieval chunks that were used as context for each LLM agent section. Each entry shows the chunk’s relative file path, chunk index, and a code snippet (line-clamped to 6 lines). This panel supports debugging and transparency by linking each analysis section to its source evidence.

6.7 Frontend Testing with Vitest

The frontend testing infrastructure uses Vitest with `@testing-library/react` and a `jsdom` environment. A shared utility module (`reportUtils.ts`) was extracted from 12 duplicate local copies across section-card components. The test suite (`utils.test.ts`) covers 12 test cases for the `normConf()` confidence normaliser and `toErrorMessage()` error formatter, validating edge cases such as undefined inputs, unrecognised strings, and non-Error thrown values.

Chapter 7

Evaluation and Results

7.1 Evaluation Methodology

CodeSpectra is evaluated across three dimensions using its own codebase (336 files spanning Python, TypeScript, and C++) as the primary test subject. This self-hosting approach provides a highly practical benchmark, allow reliable verify ground truth.

Retrieval Quality is measured via Precision@ K against 20 manually annotated information-seeking queries to assess the accuracy of the context provided to the LLM.

Agent Output Quality is human-evaluated on a 1–5 scale, focusing on factual accuracy, completeness, and schema compliance.

Runtime Performance is tracked via wall-clock time (`time.perf_counter()`) at each pipeline boundary. Results are averaged over three warmed-cache runs on a standard development machine (Windows 11, AMD Ryzen 7, 16GB RAM, NVMe SSD).

7.2 Retrieval Quality

Table 7.1 compares two retrieval configurations: a lexical-only baseline that ranks files by BM25-style term overlap alone, and the hybrid configuration that blends lexical overlap with the graph centrality signal produced by `compute_scores`.

Table 7.1: Retrieval precision comparison on CodeSpectra repository (20-query set).

Method	P@5	P@10	Coverage
Lexical only	0.52	0.48	61%
Lexical + structural	0.71	0.64	78%

Integrating the structural centrality signal improves P@5 by 0.19 (+37%) and coverage by 17 percentage points. These gains are most pronounced for abstract queries (e.g., “pipeline orchestration”), where target files lack lexical overlap but exhibit high graph centrality. Conversely, narrow implementation queries (e.g., “Tarjan SCC”) see marginal benefit, as exact keyword matches already ensure top lexical rankings.

Overall, hybrid retrieval recovers 78% of relevant ground-truth files in the top 10. The remaining 22% of missed files are predominantly dependency-graph leaf nodes lacking distinctive vocabulary—a limitation that motivates the future integration of semantic embeddings (detailed in Section 8.2).

7.3 Agent Output Quality

Table 7.2 reports the human evaluation scores for each agent when run against the CodeSpectra repository using GPT-5.3-Chat-Latest as the BYOK LLM provider. All runs used the same snapshot and identical model configuration to ensure comparability.

Table 7.2: Agent output quality scores (human evaluation, 1–5 scale).

Agent	Section	Accuracy	Completeness	Clarity
A	Project Identity	4.5	4.2	4.8
B	Architecture	4.1	3.8	4.7
C	Structure	4.3	4.0	4.7
D	Conventions	3.9	3.7	4.3
E	Violations	3.5	3.2	4.2
F	Feature Map	3.8	3.5	4.6
G	Important Files	4.4	4.1	4.8
H	Onboarding	4.0	3.8	4.7
I	Glossary	4.2	4.0	4.9
J	Risk	3.7	3.4	4.5
K	Auditor	3.6	3.6	3.4
Mean		3.91	3.75	4.63

The evaluation focuses on three qualitative dimensions: **Accuracy** (factual correctness against the codebase), **Completeness** (coverage of key components), and **Clarity** (readability, conciseness, and effective use of formatting). The consistently high Clarity

scores reflect the underlying LLM’s strong natural language generation and formatting capabilities.

Notably, **Schema Compliance** is excluded from the 1–5 scale, as the system guarantees a 100% pass rate. The base model rarely triggers the 3-attempt JSON repair fallback, demonstrating robust native instruction-following.

Factual Accuracy peaks for Agents A (4.5) and G (4.4), whose targeted inputs (documentation and centrality points) minimize confabulation. Conversely, Agents E (3.5) and K (3.6) score lowest: Agent E occasionally misidentifies idiomatic patterns as violations due to a lack of broader architectural context, while Agent K struggles with the complex meta-evaluation of other agents’ outputs.

Completeness consistently trails accuracy by ~ 0.2 points, indicating errors of omission rather than commission (i.e., outputs are accurate, but not exhaustive). This is an expected limitation of the strict retrieval budget (top-10 files), motivating the future adoption of larger context windows.

7.4 Performance Benchmarks

7.4.1 Indexing Pipeline Performance

Table 7.3 reports wall-clock times for each stage of the indexing pipeline on the CodeSpectra repository (>300 files). “First run” represents a cold start with no existing index artefacts; “Cached” represents a subsequent run on the same snapshot, where each stage detects that its output is already present and skips computation.

Table 7.3: Indexing pipeline timing on CodeSpectra repository (>300 files).

Stage	First run (ms)	Cached (ms)
Manifest build	420	< 5
Symbol extraction	1840	< 5
Graph build	380	< 5
Retrieval index build	2100	< 5
Total	4740	<20

The dominant first-run cost is the retrieval index build (2 100 ms), which processes the extracted symbol table and source chunks to construct the inverted token index.

Symbol extraction is the second costliest stage (1 840 ms), driven primarily by the time required to load and initialise Tree-sitter grammar bindings for each detected language — a one-time cost that Tree-sitter’s own grammar caching in `_loaders.py` amortises across files of the same language.

On subsequent runs against the same snapshot, all four stages complete in under 5 ms each (a hash comparison and a cache hit), reducing the total indexing overhead to under 20 ms. This makes repeat analysis — for example, changing the LLM model without modifying any source files — nearly instantaneous from the user’s perspective.

7.4.2 Native C++ vs Python Fallback

Table 7.4 compares the execution time of the six native C++ functions against estimated equivalent pure-Python implementations on a 500-node synthetic dependency graph. The Python estimates are derived from profiling the fallback implementations with `cProfile` on the same graph structure.

Table 7.4: C++ native vs Python performance (500-node graph / 300-file keyword scan).

Function	C++ (ms)	Python est. (ms)	Speedup
<code>compute_scores</code>	3	47	$\approx 15\times$
<code>compute_scc</code>	8	120	$\approx 15\times$
<code>scan_keywords_bulk</code>	12	190	$\approx 16\times$
<code>expand_neighbors</code>	2	28	$\approx 14\times$
<code>compute_louvain</code>	5	85	$\approx 17\times$
<code>rank_and_budget</code>	1	12	$\approx 12\times$

All six functions achieve approximately 12–17 $\times$ speedup. The consistency of this speedup ratio across functions of different algorithmic character (graph traversal, string scanning, community detection) suggests that the dominant factor is not algorithmic — Python’s implementations are also $O(V + E)$ or $O(n)$ — but rather the per-instruction overhead of the CPython interpreter, where each Python bytecode instruction incurs several hundred nanoseconds of overhead regardless of the operation being performed.

At the scale of CodeSpectra’s own repository (336 files, roughly 280 dependency edges), the absolute time saved by the native module is modest (tens of milliseconds). The benefit compounds on larger repositories: a 2 000-file repository with 1 500 edges

would push the Python-fallback graph operations into the multi-second range during a cold-start analysis, where they would become noticeable to the user.

7.4.3 LLM Pipeline Timing

The LLM analysis pipeline is the dominant cost centre by far. Table 7.5 summarises wall-clock time for the full twelve-agent pipeline on the CodeSpectra repository using GPT-4o-mini via the OpenAI API.

Table 7.5: LLM pipeline timing on CodeSpectra repository (GPT-4o-mini).

Run type	Wall-clock time	Notes
First run (no cache)	$\approx 45\text{s}$	Wave 0 $\approx 18\text{s}$, Wave 1 $\approx 14\text{s}$
Second run (cache hit)	$\approx 0.3\text{s}$	Report replayed from DB

The 38-second first-run time is dominated by Wave 0 (agents A, C, D, G, I, J executing in parallel, approximately 18 seconds) and Wave 1 (agents B, E, F executing in parallel after Wave 0 completes, approximately 14 seconds). Wave 2 (agents H and K) accounts for the remaining approximately 6 seconds. The three-wave structure ensures that no agent waits longer than necessary for its dependencies: Agent B, which requires Agent A’s identity output, begins as soon as Agent A completes rather than waiting for all Wave 0 agents to finish.

On the second run, the pipeline detects an existing report for the same (`snapshot_id`, `model_id`) combination in the database and replays the cached section events without invoking any LLM. The 0.3-second overhead is attributable to JSON deserialisation and event streaming latency, not computation.

7.5 Discussion

7.5.1 Strengths

No GPU required for indexing. CodeSpectra’s lexical-structural hybrid retrieval runs entirely on CPU, bypassing the need for dense vector embeddings [7]. Developers can achieve full functionality on standard laptops without managing CUDA drivers, embedding weights, or vector databases.

Interpretable retrieval scoring. The hybrid score combines two observable signals: term overlap and graph centrality. This transparency simplifies debugging; missing relevant files can be easily diagnosed and corrected by adjusting the query or blend weights.

Full local-first privacy. Integrating with local models (e.g., Ollama, LM Studio) ensures no source code leaves the developer’s machine. This satisfies strict IP and regulatory constraints that prohibit external API calls.

Consistent structured outputs. Pydantic validation and the 3-attempt repair loop guarantee structurally consistent JSON reports. This reliability makes the outputs suitable for both human review and programmatic consumption (e.g., diffing, export pipelines).

7.5.2 Weaknesses and Limitations

Model-dependent output quality. Output depth and accuracy depend heavily on the underlying LLM. While sub-7B models yield valid JSON via the repair loop, their analysis is often shallow. Domain-specific models like CodeLlama [8] or DeepSeek-Coder [6] (7B) outperform general-purpose equivalents but still trail GPT-4o-mini in completeness.

No semantic retrieval. Purely lexical retrieval struggles with semantic synonyms (e.g., “authentication” vs. `auth_service.py`). Adding an optional embedding-based signal (e.g., CodeBERT [9]) would resolve this vocabulary mismatch, albeit at the cost of additional inference time and model downloads.

Graph centrality as a proxy. Import centrality does not perfectly equate to structural importance. Ubiquitous utility modules (e.g., logging wrappers) receive artificially high scores despite low relevance. Future filtering of known utility patterns could mitigate this noise.

Single-repository scope. The architecture currently assumes a single, self-contained repository. It cannot unify dependency graphs for monorepos with independent projects or microservices that communicate over network boundaries rather than source-level imports.

Chapter 8

Conclusion and Future Work

8.1 Summary of Contributions

CodeSpectra demonstrates that deep codebase intelligence can be delivered as a fully local, privacy-preserving desktop application without relying on cloud infrastructure. The core insight is that structural signals (dependency graphs, symbol tables) combined with lexical content create a hybrid retrieval model that rivals expert intuition—without requiring dense embeddings.

The primary technical contributions are:

1. **Hybrid Lexical-Structural Retrieval:** Combines BM25-style term overlap [10] with graph centrality to retrieve relevant context across hundreds of files without a GPU or vector database.
2. **Boundary-Aware Chunking:** Expands code chunks to true function boundaries (via brace/indentation analysis), drastically reducing context-fragmentation hallucinations.
3. **12-Agent Orchestration Pipeline:** A Haystack-powered async DAG [2] that maximizes parallel execution. Each agent features distinct retrieval strategies, fixed JSON schemas, and a robust 3-attempt repair loop.
4. **Native C++ Acceleration:** Pybind11 [4] integration for heavy graph computations, achieving a 14–16× speedup over pure Python.
5. **Two-Level Caching:** Index-level and agent-result caching that drops re-analysis costs to near-zero for unchanged code.

8.2 Future Work

To further expand CodeSpectra’s capabilities, future development will focus on the following areas:

- **Semantic Retrieval with Optional Embeddings:** Integrating a local, quantized embedding model (e.g., CodeBERT [9]) to capture semantic synonyms, blending it with the current hybrid score for improved recall on abstract queries.
- **Incremental Re-indexing:** Leveraging `git diff` to process only modified files and affected graph nodes, reducing cold-start indexing times by an order of magnitude.
- **Evaluation Harness and Dataset:** Building a multi-language dataset with expert-annotated ground truths to systematically tune blend weights and centrality coefficients via $\text{Precision}@K$.
- **Multi-Repository Analysis:** Extending the graph model to map cross-repository boundaries in microservices via API call sites and shared schemas.
- **Visual Diff UI:** Enhancing the frontend to render side-by-side, colour-coded comparisons of report sections over time.
- **Plugin Architecture:** Enabling developers to register custom agents (e.g., OWASP vulnerability scanners, compliance auditors) without altering the core system, drawing inspiration from LangChain [11] and LlamaIndex [12].
- **Proactive Generative Features:** Expanding beyond read-only analysis to suggest code completions, generate localized patches, and formulate execution plans directly from task or ticket descriptions.
- **Interactive Conversational AI:** Integrating a chat-based interface that leverages the locally indexed repository context, allowing developers to ask ad-hoc, semantic questions about the codebase in real-time.

8.3 Concluding Remarks

CodeSpectra proves that structure-aware retrieval—treating the import graph topology as a primary signal rather than an afterthought—is a highly effective, deployable alternative to pure embedding-based search. The 37% relative improvement in $P@5$

precision (Section 7.2) underscores that structure captures a dimension of file relevance that text alone misses.

By pairing this retrieval foundation with strict schema enforcement and a model-agnostic architecture, CodeSpectra creates a durable platform. A pipeline that yields solid results with today's 7B local models will passively output richer insights as next-generation models emerge. Ultimately, local-first, structured-output codebase intelligence stands as a vital and powerful complement to conversational AI coding assistants.

Appendix A

Agent JSON Output Schemas

This appendix lists the JSON output schemas for each analysis agent. All schemas are enforced at runtime via the 3-attempt repair loop described in Section 5.2.1.

A.1 Agent A — Project Identity

```
1 {  
2   "repo_name":      "string",  
3   "domain":         "string",  
4   "runtime_type":   "string",  
5   "purpose":        "string",  
6   "business_context": "string",  
7   "tech_stack":     ["string"],  
8   "evidence_files": ["string"],  
9   "blind_spots":    ["string"],  
10  "confidence":      "high|medium|low"  
11 }
```

Listing A.1: Section A output schema.

A.2 Agent B — Architecture

```
1 {  
2   "main_layers": ["string"],  
3   "frameworks": ["string"],  
4   "entrypoints": ["string"],  
5   "main_services": [  
6     {"name": "string", "path": "string", "role": "string"}  
7   ]  
8 }
```

```
7   ],
8   "external_integrations": ["string"],
9   "config_sources": ["string"],
10  "database_hints": ["string"],
11  "confidence": "high|medium|low",
12  "evidence_files": ["string"],
13  "blind_spots": ["string"]
14 }
```

Listing A.2: Section B output schema.

A.3 Agent C — Structure

```
1  {
2    "folders": [
3      {
4        "path": "string",
5        "role":
6          "domain|infrastructure|delivery|shared|test|generated|unknown",
7        "description": "string"
8      }
9    ],
10   "summary": "string",
11   "confidence": "high|medium|low",
12   "evidence_files": ["string"],
13   "blind_spots": ["string"]
14 }
```

Listing A.3: Section C output schema.

A.4 Agent D — Conventions

```
1  {
2    "naming_style": {"description": "string", "evidence_files": ["string"]},
3    "error_handling": {"description": "string", "evidence_files":
4      ["string"]},
5    "async_style": {"description": "string", "evidence_files": ["string"]},
6    "di_style": {"description": "string", "evidence_files": ["string"]},
```



```
6   "class_vs_functional": {"description": "string", "evidence_files":  
7     ["string"]},  
8   "test_style": {"description": "string", "evidence_files": ["string"]},  
9   "signals": [  
10     {"pattern": "string", "category": "string", "evidence_files":  
11       ["string"]}  
12   ],  
13   "confidence": "high|medium|low",  
14   "evidence_files": ["string"],  
15   "blind_spots": ["string"]  
16 }
```

Listing A.4: Section D output schema.

A.5 Agent E — Violations

```
1 {  
2   "rules": [  
3     {  
4       "rule": "string",  
5       "strength": "strong|suspected|weak",  
6       "rationale": "string",  
7       "evidence_files": ["string"]  
8     }  
9   ],  
10  "violations": [  
11    {  
12      "rule_broken": "string",  
13      "location": "string",  
14      "severity": "high|medium|low"  
15    }  
16  ],  
17  "confidence": "high|medium|low",  
18  "evidence_files": ["string"],  
19  "blind_spots": ["string"]  
20 }
```

Listing A.5: Section E output schema.

A.6 Agent F — Feature Map

```
1 {
2   "features": [
3     {
4       "name": "string",
5       "description": "string",
6       "entrypoint": "string",
7       "key_files": ["string"],
8       "data_path": "string",
9       "tests": ["string"],
10      "reading_order": ["string"]
11    }
12  ],
13  "confidence": "high|medium|low",
14  "evidence_files": ["string"],
15  "blind_spots": ["string"]
16 }
```

Listing A.6: Section F output schema.

A.7 Agent G — Important Files

```
1 {
2   "entrypoint": {"file": "string", "reason": "string"},
3   "backbone": {"file": "string", "reason": "string"},
4   "critical_config": {"file": "string", "reason": "string"},
5   "highest_centrality": {"file": "string", "reason": "string"},
6   "most_dangerous_to_touch": {"file": "string", "reason": "string"},
7   "read_first": {"file": "string", "reason": "string"},
8   "other_important": [{"file": "string", "reason": "string"}],
9   "confidence": "high|medium|low",
10  "evidence_files": ["string"],
11  "blind_spots": ["string"]
12 }
```

Listing A.7: Section G output schema.

A.8 Agent H — Onboarding

```
1 {
2   "steps": [
3     {
4       "order": 1,
5       "file": "string",
6       "goal": "string",
7       "outcome": "string"
8     }
9   ],
10  "total_estimated_minutes": 30,
11  "confidence": "high|medium|low",
12  "evidence_files": ["string"],
13  "blind_spots": ["string"]
14 }
```

Listing A.8: Section H output schema.

A.9 Agent I — Glossary

```
1 {
2   "terms": [
3     {"term": "string", "definition": "string", "evidence_files":
4       ["string"]}
5   ],
6   "confidence": "high|medium|low",
7   "blind_spots": ["string"]
8 }
```

Listing A.9: Section I output schema.

A.10 Agent J — Risk

```
1 {
2   "findings": [
3     {
4       "category":
5         "god_object|circular_import|todo_hotspot|test_gap|blast_radius|config_risk|heavy_bra
```

```
5     "severity": "high|medium|low",
6     "title": "string",
7     "rationale": "string",
8     "evidence": ["string"]
9   }
10 ],
11 "summary": "string",
12 "confidence": "high|medium|low",
13 "evidence_files": ["string"],
14 "blind_spots": ["string"]
15 }
```

Listing A.10: Section J output schema.

A.11 Agent K — Auditor

```
1 {
2   "overall_confidence": "high|medium|low",
3   "section_scores": {
4     "A": "high|medium|low",
5     "B": "high|medium|low",
6     "C": "high|medium|low",
7     "D": "high|medium|low",
8     "E": "high|medium|low",
9     "F": "high|medium|low",
10    "G": "high|medium|low",
11    "H": "high|medium|low",
12    "I": "high|medium|low",
13    "J": "high|medium|low"
14  },
15   "weakest_sections": ["string"],
16   "coverage_percentage": 0.0,
17   "notes": "string",
18   "blind_spots": ["string"]
19 }
```

Listing A.11: Section K output schema.

A.12 Agent L — Synthesis

```
1 {  
2   "executive_summary": "string",  
3   "architecture_narrative": "string",  
4   "tech_stack_snapshot": "string",  
5   "developer_quickstart": "string",  
6   "conventions_digest": "string",  
7   "risk_highlights": "string",  
8   "reading_path": "string",  
9   "confidence": "high|medium|low"  
10 }
```

Listing A.12: Section L output schema.

Appendix B

Backend API Reference

The CodeSpectra Python backend exposes a RESTful API on `localhost` (dynamic port). All endpoints return JSON. This appendix lists the key endpoints.

B.1 Analysis Endpoints

Table B.1: Analysis API endpoints.

Method	Path	Description
POST	<code>/api/analysis/start</code>	Start analysis job. Body: <code>StartAnalysisRequest</code> .
GET	<code>/api/analysis/reports</code>	List saved reports (filter by <code>repo_id</code>).
GET	<code>/api/analysis/reports/{id}</code>	Get single report.
DELETE	<code>/api/analysis/reports/{id}</code>	Delete report.
POST	<code>/api/analysis/compare</code>	Diff two reports. Body: <code>{report_id_a, report_id_b}</code> .
POST	<code>/api/analysis/rerun_section</code>	Re-run a single agent section.
GET	<code>/api/analysis/reports/{id}/export/audit</code>	Export Section K to Markdown.
GET	<code>/api/analysis/reports/{id}/export/markdown</code>	Export full report to Markdown.

StartAnalysisRequest (current contract):

```
1 {
2   "repo_id": "string",
3   "snapshot_id": "string",
4   "scan_mode": "quick|full",
5   "privacy_mode": "strict_local|byok_cloud",
6   "provider_id": "string",
7   "model_id": "string",
8   "force_rerun": false,
9   "large_codebase_mode": false
10 }
```

Listing B.1: POST /api/analysis/start request body.

Note. /api/analysis/events/{job_id} returns a stream-like polling payload:

```
1 {
2   "events": [
3     {
4       "type": "analysis:section_done",
5       "section": "A|...|L",
6       "status": "done|error",
7       "duration_ms": 1234,
8       "data": {"...": "..."},
9       "error": null
10    }
11  ],
12   "job_done": false
13 }
```

Listing B.2: GET /api/analysis/events/{job_id} response shape.

B.2 Repository and Snapshot Endpoints

Local repository management (/api/local-repo)

Table B.2: Local repository endpoints.

Method	Path	Description
POST	/api/local-repo/validate	Validate a folder before adding as managed repo.
GET	/api/local-repo/	List managed local repositories.
POST	/api/local-repo/	Add local repository by path.
POST	/api/local-repo/clone	Clone from URL into managed repos directory.
DELETE	/api/local-repo/{repo_id}	Remove managed repo entry (and associated metadata).
POST	/api/local-repo/{repo_id}/revalidate	Re-scan/refresh repository metadata.
GET	/api/local-repo/{repo_id}/branches	List Git branches (refresh optional).
POST	/api/local-repo/{repo_id}/branch	Set selected branch.
POST	/api/local-repo/{repo_id}/active-snapshot	Mark active snapshot for UI/runtime.
POST	/api/local-repo/{repo_id}/settings	Update repo-specific settings.
GET	/api/local-repo/{repo_id}/estimate-file-count	Fast file-count estimate for UX warnings.

Snapshot lifecycle (/api/sync)

Table B.3: Snapshot preparation endpoints.

Method	Path	Description
POST	/api/sync/prepare	Prepare a repository snapshot (copy/metadata capture).
GET	/api/sync/snapshot/{snapshot_id}	Get one snapshot metadata record.
GET	/api/sync/repo/{repo_id}	List snapshots of a repository (limit supported).
DELETE	/api/sync/snapshot/{snapshot_id}	Delete snapshot and derived artifacts.

Manifest and symbol inventory

Table B.4: Manifest and repo-map endpoints.

Method	Path	Description
POST	/api/manifest/build	Build manifest file inventory for a snapshot.
GET	/api/manifest/preview/{snapshot_id}	Preview first N manifest files.
GET	/api/manifest/tree/{snapshot_id}	Return folder-tree structure from manifest.
GET	/api/manifest/file/{snapshot_id}	Read one file by path (size-limited).
POST	/api/repo-map/build	Parse symbols/imports and store repo map.
GET	/api/repo-map/summary/{snapshot_id}	Repo-map summary metrics.
GET	/api/repo-map/symbols/{snapshot_id}	List symbols (limit, path_prefix).
GET	/api/repo-map/search/{snapshot_id}	Search symbols by keyword q.
GET	/api/repo-map/export/{snapshot_id}	Export symbols to CSV payload.

B.3 Provider Configuration Endpoints

Table B.5: Provider and model-connector endpoints.

Method	Path	Description
GET	/api/provider/	List configured providers.
POST	/api/provider/	Create provider (kind, URL, model, capabilities, API key).
GET	/api/provider/{provider_id}	Get provider details.
PUT	/api/provider/{provider_id}	Update provider configuration.
DELETE	/api/provider/{provider_id}	Delete provider.
POST	/api/provider/{provider_id}/test	Probe provider connectivity and auth.
GET	/api/provider/{provider_id}/models	List remote models available from provider.

Provider error mapping. Domain-level provider errors are normalized into HTTP statuses: 401 auth failure, 404 model not found, 422 context limit exceeded, 429 rate limited, 503 timeout/connection refused, 502 unknown upstream error.

B.4 Retrieval Debug Endpoints

Structural graph, community detection, and retrieval

Table B.6: Graph, community, and retrieval endpoints.

Method	Path	Description
POST	/api/graph/build	Build import graph, central-ity summary, and trigger back-ground community detection.
GET	/api/graph/summary/{snapshot_id}	Return graph summary, top central files, and native toolchain status.
GET	/api/graph/edges/{snapshot_id}	List graph edges (limit , internal_only supported).
GET	/api/graph/neighbors/{snapshot_id}	Expand neighbor subgraph from a seed path (hops , limit).
GET	/api/graph/communities/{snapshot_id}	List all Louvain communities with node index.
GET	/api/graph/community/{snapshot_id}	Single node's community ID and all members (path query param).
GET	/api/graph/cycles/{snapshot_id}	SCCs with ≥ 2 nodes (circular import candidates).
GET	/api/graph/export/{snapshot_id}	Full JSON export: nodes, edges, communities, community edges, cycles, test files.
POST	/api/retrieval/build-index	Build chunk index and BM25 IDF statistics for retrieval.
POST	/api/retrieval/retrieve	Retrieve evidence bundle (delegates to two-stage pipeline).
POST	/api/retrieval/retrieve-two-stage	Two-stage retrieval: BM25 $\rightarrow$ graph expansion $\rightarrow$ rank and budget.
POST	/api/retrieval/compare	Compare hybrid vs vectorless retrieval behavior.

Fallback behavior (runtime). When the native C++ module is unavailable, graph scoring, neighbor expansion, community detection, and two-stage ranking continue via Python fallback implementations, so all graph and retrieval endpoints remain functional (with lower performance).

System and utility endpoints

Table B.7: Utility endpoints (`/api/app`, `/api/consent`, `/api/workspace`, `/api/job`).

Method	Path	Description
GET	<code>/api/app/health</code>	Backend liveness and version string.
GET	<code>/api/app/diagnostics</code>	Runtime diagnostics (Python version, native function availability).
GET	<code>/api/app/git-config</code>	Read persisted Git SSH key path.
PUT	<code>/api/app/git-config</code>	Update persisted Git SSH key path.
GET	<code>/api/consent/cloud</code>	Read cloud-consent flag used by BYOK mode.
POST	<code>/api/consent/cloud</code>	Set cloud-consent flag.
GET	<code>/api/workspace/</code>	List workspaces.
POST	<code>/api/workspace/</code>	Create workspace (with optional description).
GET	<code>/api/workspace/{workspace_id}</code>	Get single workspace (includes description).
PUT	<code>/api/workspace/{workspace_id}</code>	Update workspace name and description.
DELETE	<code>/api/workspace/{workspace_id}</code>	Delete workspace.
POST	<code>/api/job/</code>	Create generic background job.
GET	<code>/api/job/{job_id}</code>	Get job status/progress.
POST	<code>/api/job/{job_id}/cancel</code>	Request cancellation.
GET	<code>/api/job/repo/{repo_id}</code>	List jobs for a repository.
GET	<code>/api/job/</code>	List recent jobs.

References

- [1] J. Parrish et al., *Aiosqlite: Async bridge to the standard sqlite3 module*, Accessed: April 2026, 2019. [Online]. Available: <https://github.com/omnilib/aiosqlite>
- [2] deepset GmbH, *Haystack: Open-source LLM framework*, Accessed: April 2026, 2024. [Online]. Available: <https://haystack.deepset.ai/>
- [3] M. Brunsfeld et al., *Tree-sitter: An incremental parsing system for programming tools*, Accessed: April 2026, 2024. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>
- [4] W. Jakob, J. Rhineland, and D. Moldovan, *Pybind11 — seamless operability between C++11 and Python*, Accessed: April 2026, 2016. [Online]. Available: <https://github.com/pybind/pybind11>
- [5] R. E. Tarjan, “Depth-first search and linear graph algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [6] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [7] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [8] B. Rozière et al., “Code Llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [9] Z. Feng et al., “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.
- [10] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [11] H. Chase et al., *LangChain: Building applications with LLMs through composability*, Accessed: April 2026, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>

-
- [12] J. Liu et al., *LlamaIndex: Data framework for LLM applications*, Accessed: April 2026, 2022. [Online]. Available: https://github.com/run-llama/llama_index